NTNU
Norwegian University of Science and Technology (NTNU)
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

NTNU

Master's Thesis

Candidate:	**Bukueva Elena**
Course:	**TTK4900 Master's Thesis**
Title (English):	**Compiler-Aware Model Optimization for Edge AI**
Title (Norwegian):	**Kompilerbevisst optimaliseringfor Edge-AI**

Thesis description: Efficient deployment of deep learning models on embedded and edge platforms depends on both model-level and system-level factors. In addition to model design, inference performance is influenced by compiler support, runtime behavior, hardware characteristics, numerical precision, and optimization techniques such as quantizatio n and pruning. This thesis studies compiler-aware model optimization for edge AI, with focus on real-time object detection. The aim is to evaluate how different deployment configurations affect accuracy, latency, and resource usage, and to identify the main bottlenecks that limit efficient execution on target hardware.

The tasks will be:

1. Develop a framework for evaluating deep learning models across deployment configurations in terms of accuracy, latency, and resource usage.
2. Explore model optimization techniques to improve computational efficiency while maintaining acceptable performance.
3. Analyze optimized models on target hardware, identify bottlenecks, and investigate ways to improve execution efficiency.

Start date:	15. January, 2026
Due date:	08. June, 2026
Thesis performed at:	Department of Engineering Cybernetics
Supervisor:	Professor Geir Mathisen, Dept. of Eng. Cybernetics

Abstract

Sammendrag

Contents

Abbreviations

1 Introduction

1.1 Motivation

1.2 Limitations

1.3 Contributions

Within these limitations, the main contributions of this thesis are:

- **:**
- **:**
- **:**
- **:**
- **:**

1.4 Thesis Structure

The remainder of this report is organised as follows:

- **Chapter 2**
- **Chapter 3**
- **Chapter 4**
- **Chapter 5**
- **Chapter 6**
- **Chapter 7**
- **Chapter 8**
- **Chapter 9**

- **Chapter 10**
- **Appendix A**
- **Appendix B**

2 Literature Study

This chapter is currently in draft form and requires further structuring, formatting, and completion of the literature review. Its final revision is planned for the concluding phase of the thesis work.

2.1 Scope of the chapter

This chapter reviews the literature and technical background relevant to the objectives of this thesis. The focus is not on providing an exhaustive survey of all deep learning optimization methods, but rather on summarizing the concepts, systems, and prior work that are directly related to compiler-aware optimization of object detection models for edge deployment.

The review is structured around the main themes that support the experimental part of the thesis. First, it introduces object detection models, with particular emphasis on the YOLO family and the YOLOv8 architecture used in this work. Second, it discusses the main challenges of deploying deep learning models on edge devices, including latency constraints, limited memory bandwidth, reduced compute resources, and the importance of efficient inference at small batch sizes. Third, it reviews optimization techniques that are central to this thesis, namely mixed-precision inference, quantization, pruning, and selected forms of structural simplification.

In addition, the chapter examines machine learning compilers and inference runtimes as the software layer through which deployment optimization is realized. Particular attention is given to Apache TVM and TensorRT, since these frameworks are used in the experimental study and represent two different approaches to inference optimization: a flexible compiler-oriented stack and a highly optimized vendor-specific deployment framework. Finally, the review includes performance-oriented concepts such as operator fusion, memory traffic reduction, and the distinction between compute-bound and memory-bound execution, as these are necessary for interpreting the results presented later in the thesis.

The literature covered in this chapter is therefore selected according to its relevance to the research questions and experiments of this work. Topics that are not directly connected to the implemented methods or evaluated deployment scenarios are discussed only briefly or omitted. This selective approach helps maintain focus on the techniques and systems that are most important for understanding the methodology and the experimental findings of the thesis.

2.2 Introduction to Object Detection with Deep Learning Models

TODO: Description of common object detection models and introduction to the the YOLO family

2.3 YOLOv8 Architecture

YOLOv8 is a one-stage convolutional object detector introduced by Ultralytics in 2023 [15] as part of the YOLO family of real-time vision models. Its design follows the standard decomposition of modern detectors into a backbone, a neck, and a prediction head, while introducing several architectural refinements compared with earlier YOLO generations. In particular, YOLOv8 employs C2f feature extraction blocks, an anchor-free split detection head, and an efficient multiscale feature aggregation pipeline. These design choices aim to improve the trade-off between detection accuracy and inference speed while maintaining practical deployability across heterogeneous hardware platforms. According to the official Ultralytics implementation, the standard detection family includes five model scales, ranging from YOLOv8n to YOLOv8x, where the nano variant contains approximately 3.2 million parameters and requires about 8.7–8.9 GFLOPs at an input resolution of 640×640.

Overall structure of YOLOv8 The overall YOLOv8 architecture is illustrated in Figure 2.1. The figure also highlights the scaling variables used to derive the different model variants, which determine the network depth, width, and overall computational complexity.

The standard YOLOv8 detection network consists of three main stages. The backbone extracts hierarchical visual features from the input image. The neck fuses semantically rich low-resolution features with spatially precise higher-resolution features. Finally, the prediction head converts the fused multiscale features into bounding box coordinates and class predictions. In the reference YOLOv8 configuration, the backbone progressively downsamples the input image into feature maps at multiple pyramid levels, while the head generates predictions at three output scales commonly associated with small, medium, and large objects. In the default P5 configuration, predictions are produced from feature levels P3/8, P4/16, and P5/32.

The backbone begins with strided convolutional layers that reduce spatial resolution while increasing channel depth. These stages are interleaved with C2f blocks, which are

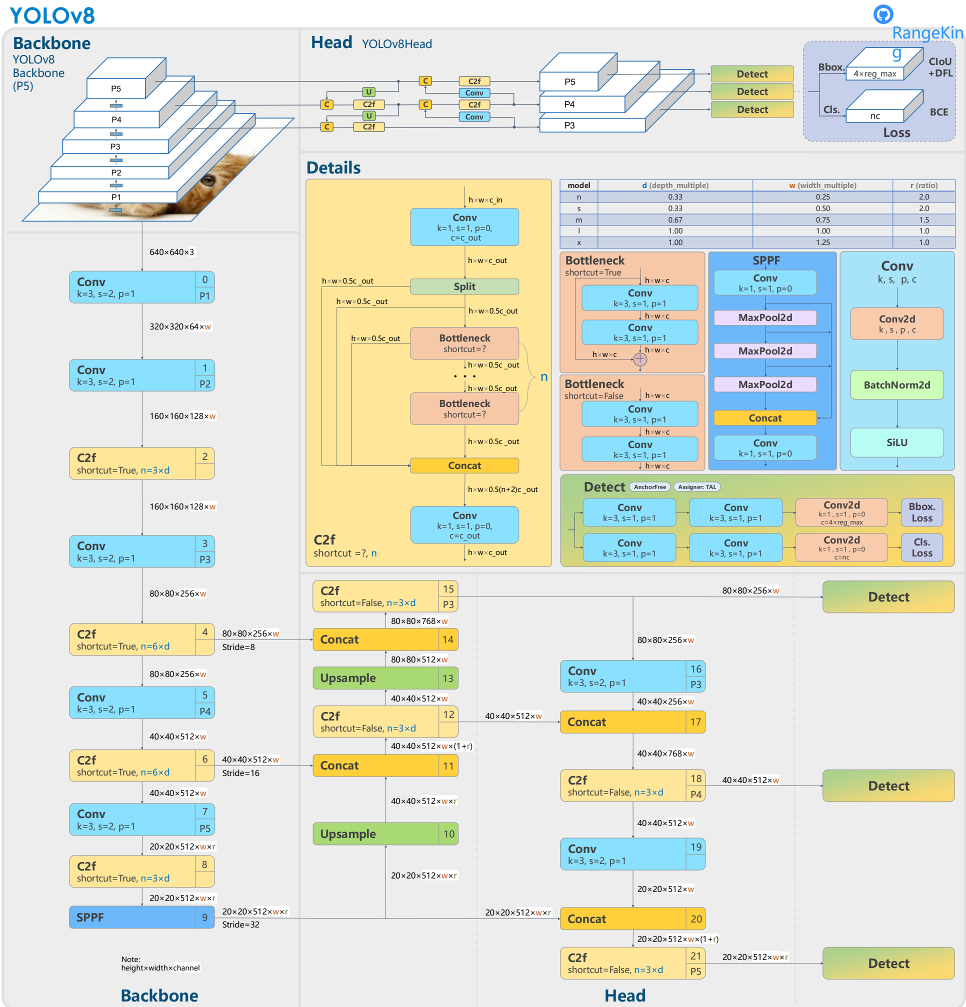

Figure 2.1: Overall YOLOv8 architecture and model scaling variables [35]

described in the Ultralytics reference implementation as a “faster implementation of CSP bottleneck with 2 convolutions.” The main purpose of these blocks is to improve gradient flow and feature reuse while maintaining computational efficiency. At the deepest stage of the network, YOLOv8 employs an SPPF (Spatial Pyramid Pooling – Fast) module, which increases the effective receptive field and improves multiscale context aggregation with relatively low computational overhead.

Overall, the main backbone components of YOLOv8 are:

- convolutional blocks with Batch Normalization and SiLU activation,
- C2f feature extraction blocks,
- the SPPF module.

The C2f block itself is composed of several bottleneck units. Conceptually, such a block first reduces the number of channels, then performs spatial feature transformation, and finally combines intermediate representations to preserve useful information. This design improves feature propagation and reduces redundant computation while retaining informative representations. Compared with earlier CSP-style blocks, C2f enables richer feature reuse and more efficient gradient flow.

The neck follows a PAN-FPN style design. In practice, this means that feature maps are upsampled, concatenated with earlier backbone features, and then processed again using C2f blocks. This combination of top-down and bottom-up feature fusion is central to YOLOv8's ability to detect objects across multiple scales. The reference YAML architecture repeatedly uses Upsample, Concat, and C2f blocks before the final Detect module. Such multiscale aggregation is particularly important for real-world datasets, where object size can vary significantly across scenes.

The prediction head in YOLOv8 is both decoupled and anchor-free. Ultralytics describes YOLOv8 as using an anchor-free split detection head, meaning that predictions are not tied to a predefined set of anchor box shapes. Instead, the detector predicts bounding boxes and class scores directly from the feature maps. Compared with older anchor-based YOLO variants, this simplifies the detection pipeline and removes the need for dataset-specific anchor tuning.

In an anchor-free detector, each spatial location on the feature map typically predicts:

- whether an object is present,
- the class scores,
- the box coordinates or distances to the box boundaries.

The split head further separates the localization and classification tasks into distinct branches. This decoupling allows the model to optimize these tasks more effectively, since object localization and semantic classification impose different requirements on the learned feature representations.

Another important component of the YOLOv8 detection formulation is its use of distribution-based box regression. The Ultralytics codebase includes a DFL module and the corresponding loss implementation, reflecting the use of Distribution Focal Loss for bounding box parameterization. This improves localization precision by modeling bounding box offsets as discrete probability distributions rather than relying solely on coarse direct regression.

Relevance of YOLOv8 for this thesis For a thesis focused on compiler-aware optimization of deep learning models for edge devices, YOLOv8, and especially YOLOv8n, represents a highly suitable case study.

First, YOLOv8 offers a strong balance between detection accuracy and inference latency, which is one reason why later detectors continue to use it as a baseline. Second, YOLOv8 has a mature deployment ecosystem. Ultralytics supports export to a wide range of formats relevant for practical deployment, including ONNX, TensorRT, CoreML, OpenVINO, and TFLite. This makes the model especially attractive for evaluating compiler and runtime behavior across different hardware platforms.

In addition, YOLOv8 is not limited to object detection alone. The model family also supports segmentation, classification, pose estimation, and oriented bounding box detection within the same software ecosystem. This is particularly relevant in applied edge AI research, where systems often evolve from simple object detection toward richer scene understanding tasks. As a result, YOLOv8 provides both architectural consistency and practical flexibility.

For this thesis, YOLOv8n is of particular interest because it is small enough to run on many embedded platforms, while still being architecturally complex enough to expose realistic deployment bottlenecks. The model includes multiscale feature fusion, repeated tensor concatenations, SiLU activations, distribution-based box regression, and post-processing stages. These properties make it a meaningful test case for studying compiler optimizations, runtime scheduling, kernel efficiency, quantization behavior, and memory-system effects.

Despite being the smallest variant, YOLOv8n is not automatically optimal for every edge target. Several practical challenges arise during deployment.

First, a low parameter count does not necessarily translate into low latency on all hardware platforms. YOLOv8n still contains nontrivial tensor reshaping, concatenation, upsampling, and multiscale fusion operations. On desktop GPUs, such operations may be efficiently hidden by optimized kernels and memory systems, whereas on edge CPUs and lightweight accelerators the cost of memory movement can dominate the arithmetic workload. In such cases, the effective execution becomes memory-bound rather than compute-bound.

Second, operator support and kernel efficiency vary significantly across deployment backends. Although YOLOv8 can be exported to ONNX, TensorRT, TFLite, and other runtimes, the achieved performance depends strongly on how well the target backend supports the specific operator set and graph structure of the model. In practice, operations such as SiLU, concatenation-heavy neck structures, and certain post-processing steps may be implemented with different levels of efficiency across frameworks. As a result, there is often a noticeable gap between the theoretical FLOP count of the model and the measured end-to-end latency.

Third, post-processing introduces an additional deployment challenge. Standard

YOLOv8 detection still relies on confidence filtering and non-maximum suppression after the forward pass. On GPU-based systems, this may require transferring relatively large tensors from the device to the host, followed by post-processing on the CPU. Such steps can add non-negligible latency and complicate the analysis of true end-to-end performance.

Overall, YOLOv8 is an interesting subject for deployment-oriented research because it combines architectural complexity with competitive accuracy and speed. In particular, YOLOv8n remains highly relevant for edge AI studies: it is compact enough to be practical on constrained devices, yet sufficiently complex to reveal the impact of compiler transformations, runtime optimizations, operator fusion, and memory bottlenecks on real inference workloads.

2.4 Challenges of edge deployment

Metrics:

- **Latency**: The time needed to process one inference request (etc. picture, text query);
- **Throughput**: The number of inference requests processed per unit of time (e.g., inferences per second).
- **Power Consumption**: The energy required to perform inference
- **Memory Requirements**: The amount of RAM and storage required for the model weights and intermediate activations

With the growing interest in on-device deployment, a clear gap has emerged between the training and inference stages of machine learning models. While large models are typically trained in data centers using powerful GPUs, inference is performed across a wide variety of deployment environments. Depending on the application, inference may be subject to strict constraints on memory, latency, and power consumption, and must run on heterogeneous hardware platforms such as CPUs, GPUs, TPUs, NPUs, or FPGAs. Each of these hardware architectures differs in structure, instruction sets, memory hierarchies, and supported data types. As a result, model performance depends heavily on how well it is adapted to the target hardware.

Machine learning models are usually developed in high-level Python frameworks such as TensorFlow or PyTorch. These frameworks often use dynamic execution (for example, PyTorch's eager mode), where:

- Operations are executed immediately, line by line
- Control flow (loops, conditionals) is resolved at runtime
- Tensors, layers, and operations are created and destroyed on the fly

That same flexibility comes at a cost:

- **Python interpreter overhead:** Every operation goes through the Python interpreter, which is relatively slow.
- **Interpreter locks (e.g., the GIL):** Python's Global Interpreter Lock can limit parallel execution.
- **Object management:** Tensors and operators are Python objects that need to be created, tracked, and destroyed.
- **Framework dispatch logic:** Each operation requires the framework to decide which kernel to run, on which device, and how to manage memory.

Individually, these costs are small — but in deep neural networks with thousands of operations, they add up.

In deployment, models are usually not run inside Python at all.

Instead, they are executed by:

- C++-based runtimes
- Hardware-specific inference engines
- Precompiled binaries on accelerators (GPU, NPU, TPU, FPGA)

These runtimes:

- Execute pre-compiled computation graphs
- Call highly optimized kernels directly
- Avoid Python entirely during inference

As a result, they remove most of the overhead associated with Python and dynamic execution.

Simply moving a model out of Python does not automatically guarantee speedups.

To fully benefit, the model must be:

- Compiled into a static or semi-static graph
- Optimized (operator fusion, memory planning, quantization, layout changes)
- Adapted to the target hardware's instruction set and memory hierarchy

Without this careful compilation step, the deployed runtime may still execute inefficient kernels or suboptimal data flows.

Libraries like cuDNN and oneDNN are heavily optimized for throughput-oriented workloads, which historically means large batch sizes. Large batches increase data reuse (e.g., weights reused across many samples), which improves cache efficiency and amortizes memory access costs. With many parallel threads and large workloads, GPUs/CPUs can overlap memory accesses with computation, effectively hiding memory latency. Many optimized kernels (e.g., GEMM-based convolutions) assume enough work to justify complex tiling, blocking, and scheduling strategies. These libraries were primarily developed to support large-scale training in data centers, where batch sizes of hundreds or thousands are common. So the default optimization targets of these libraries align better with large batches than with batch-1 inference, which is preferable for Real-time setups

Frameworks like PyTorch (in eager mode) execute operation by operation, where each op is dispatched independently; intermediate tensors are materialized in memory; the framework has limited visibility beyond the current operator. Compilers, in contrast, operate on a whole computation graph. All operators and data dependencies are visible; control flow and tensor lifetimes can be analyzed globally. Compilers merge multiple operations into a single kernel to reduce memory traffic and overhead.

With a full graph, compilers can:

- Remove redundant operations
- Fold constants
- Reorder associative/commutative ops

- Eliminate dead branches or unused tensors

Furthermore, different hardware prefers different layouts, GPUs often prefer NCHW, Some accelerators prefer NHWC or blocked layouts, so compilers can choose a single optimal layout, insert minimal layout conversions, avoid unnecessary transposes.

2.5 ML compilers and inference runtimes

An ML model compiler takes a *high-level model* (PyTorch / TensorFlow / ONNX) and turns it into efficient executable code for a *specific target* (CPU, GPU, NPU, FPGA, ASIC like Hailo-8).

Stages:

- Stage 1: Frontend (Model ingestion)
  - Input: PyTorch, TensorFlow, JAX
  - Exported formats: ONNX, TorchScript, StableHLO
  - Goal:
    - Convert framework-specific graphs into a compiler IR
    - Normalize control flow, ops, shapes
  - Key tasks:
    - Canonicalize ops (e.g. many conv variants → one canonical conv)
- Stage 2: High-level graph optimizations (framework-agnostic)
- Stage 3: Lowering to tensor programs (IR → loops)
- Stage 4: Hardware-specific optimization & scheduling
- Stage 5: Code generation (backend)
- Stage 6: Runtime & execution

A typical ML execution flow involves translating a model defined in a high-level framework (like TensorFlow, PyTorch, or JAX) into a format suitable for high-performance execution on target hardware. This translation process is orchestrated by the ML compiler and runtime stack (see Fig. 2.2)

The Compiler's primary responsibility is to convert high-level representation of a DNN model to an optimized lower-level executable by the target hardware.

Main stages:

- Conversion to an intermediate representation (IR);
- Graph level optimization: operation fusion and simplification, memory planning;
- Tensor/Loop-Level IR: tiling, vectorization, memory optimization;
- Code generation: intruction selection, register allocation, target-specific code emission;

ML runtime manages the dynamic aspects of model execution such as execution orchestration (loading compiled model artifact and managing its execution), memory management (allocation of memory for intermediate tensors), interfacing with hardware devices, task dispatching, and library integration.

Examples of modern compiler and runtime stacks for machine learning include:

- **TensorFlow/XLA**, which compiles TensorFlow programs through XLA using HLO-based intermediate representations [28, 29];
- **PyTorch 2.x**, which combines TorchDynamo, AOTAutograd, and TorchInductor to capture and optimize models for efficient execution [32, 33, 31];
- **Apache TVM**, a compiler stack for optimizing models across heterogeneous targets including CPUs, GPUs, and edge-oriented accelerators [1, 2];
- **MLIR-based stacks**, such as IREE and TensorFlow MLIR, which use multi-level intermediate representations to enable flexible lowering pipelines for heterogeneous systems [20, 38, 13];
- **ONNX Runtime**, a cross-platform inference engine that supports hardware-specific execution providers for efficient deployment on diverse platforms [26, 27].

2.6 Quantization and mixed precision

2.7 Pruning and structural simplification

We can categorize studies of pruning neural networks into 8 categories:

- **Sensitivity analysis methods (loss / gradient–aware pruning)**. These methods are valuable as they prune what matters the most for the task and normally provide the highest accuracy-compression trade-off (50-90% accuracy with 1-2% accuracy drop).
  Important works in this category:
  - Taylor Pruning [23];
  - SNIP (Single-shot Network Pruning) [16];
  - GraSP (Gradient Signal Preservation) [41];
  - Movement Pruning [6]

  Although sensitivity-based pruning methods often achieve high unstructured sparsity (50–90%), such sparsity is difficult to exploit on general-purpose hardware due to irregular memory access and limited compiler support, resulting in limited real-world speedups. Consequently, structured pruning methods remain more practical for deployment across diverse hardware platforms.

- **Knowledge distillation methods (teacher–student compression)**. Knowledge Distillation (KD) is a model compression paradigm in which a large, high-capacity model (Teacher) transfers knowledge to a smaller, more efficient model (Student). Instead of training the student solely on hard labels, KD trains it to match the teacher's output behavior, typically using soft targets (probability distributions or logits). KD differs fundamentally from standard understanding of pruning: pruning removes parameters or structures, but KD transfers behavior. By doing this KD is normally does both: improves the accuracy by few percent compared to the baseline and reduces the number of parameters up to 120 times. Important works:
  - Knowledge Distillation [10];
  - DCP [43];
  - AutoCompress [19];

  Knowledge distillation is a powerful model compression paradigm that transfers predictive behavior from a large teacher model to a compact student model using soft supervision. While classical distillation operates at the output level, recent works extend this idea to intermediate representations and pruning-aware settings. Methods such as discrimination-aware channel pruning integrate auxiliary losses to preserve discriminative power during structured pruning, achieving strong accuracy retention. However, distillation-based approaches are inherently task-dependent, require access to labeled data and a strong teacher, and introduce additional training complexity, limiting their applicability in representation-centric or hardware-agnostic compression scenarios.

- **Similarity and clustering methods** Similarity and clustering–based pruning methods aim to remove redundant filters or channels rather than filters that are individually "unimportant". The core assumption is that modern CNNs are over-parameterized and contain filters whose functionality can be approximated or replaced by others with minimal impact on accuracy. Instead of ranking filters solely by magnitude or sensitivity, these methods explicitly model correlation, similarity, or representational overlap among filters. Representative methods include:
  - FilterSketch by Lin et al. [18],
  - Geometric Median Pruning by He et al. [9],
  - ThiNet by Lu et al. [21]
- **Low rank methods** are mostly theoretically grounded methods, effective in convolution-heavy models; Best methods: Show moderate speedups and faster accuracy degradation at high compression
- **Magnitude based pruning methods** are considered simple and cheap to implement models, however, they have limited compression and higher accuracy drops, compared to more complex structures; Best examples are L1 filter pruning (Li et al., 2017) [17], APoZ (Activation sparsity) [11].
- **Architectural design methods (NAS, RL search)**. examples: AMC, AutoSlim, Once-for-All (OFA)
- **Hybrid methods**
- **Quantization methods**: are not exactly pruning methods. They are usually combined with pruning. Mostly based on a transformation of weights from floating point to integer. Important works: Quantization-Aware Training (QAT) [14], Binary / Ternary Networks (XNOR, TWN) [37]

2.7.1 Magnitude based pruning

Magnitude based Pruning Methods are methods based on removing weights, reducing amount of inbound and outbound filters and nodes based on a measure of magnitude of the effect these filters have on the network. Pruning iteration is repeated until the accuracy is starting to drop. First works in this field were adopting the idea of removing the weight lower than a particular threshold.

Han et al. [8] carried out several experiments on LeNet-300-100 and Lenet-5, both trained on MNIST dataset, and showed that weight could be reduced by this method by a factor of 12 without significant drops in accuracy. Experiments based on AlexNet and VGG16 trained on ImageNet show the factor of 9 and 12 respectively. Han et al. note that L_1 regularization, a technique used in machine learning to prevent overfitting by adding a penalty term to the cost function, is better immediately after pruning, but that L_2 norm is better, when the weights of the pruned model are fine-tuned. Experiments also suggest that earlier layers are the most sensitive to pruning and that iterative pruning is better that one-shot pruning (cutting the proportion of weights in one cycle). Another observation from their work is that re-initialization of the weights is not enough to construct and accurate model. It is always better to fine-tune the weights of the pruned model.

Later work by Guo et al. [7] notes that magnitude pruning can lead to premature removal of weight that can become important given removal of other weights. They propose a method called Dynamic Network Surgery to keep track of pruned weights and restore them if they turn to be important. For AlexNet on ImageNet they reach a factor of 17 without compromising accuracy.

The other method proposed by the Frankle and Carbin (2018) is the lottery ticket hypothesis proposed by [3] and further continued by Frankle et al. (2019) in [4]. It formulates a statement, that a trained network contains a subnetwork, which can be trained to be at least as accurate as the original network using no more than the number of epochs used for training the original network. This subnetwork is a so-called winning lottery ticket. Authors test two pruning methods. The first is a magnitude pruning (p% of the weights), after which the remaining weights are reset to their initial values and are retrained. The second method is using an alternative pruning to iteratively prune $p^{\frac{1}{n}}$ of the weights in n cycles. They experimented on VGG and ResNet and on some bigger models suggested that setting of the weights to those obtained from later iteration of training can be beneficial when high levels of pruning are used (more than 30%).

Another question arises is if these subnetworks are transferable to other tasks. Hubens et al. (2020) [12] show that when a network is trained on a larger data set, such as ImageNet, and transferred and fine-tuned for a different task, pruning can result in a smaller network than if it was trained from scratch on the new task.

Despite the experiments proving that some unstructured pruning methods allow a significant drop of weights, not all hardware can process sparse weight matrices and support speed up inference. Pruning of higher granularity structures (filters and channels) or so-called structured pruning helps to avoid this problem, as many existing toolkits support it. Survey [40] suggests three main directions of structured pruning research:

- **Data dependent channel pruning methods**. A useful channel should respond differently to different inputs. A channel that barely changes for any input is potentially useless.
  Polyak & Wolf (2015) [30] propose two methods - inbound pruning (to reduce the amount of input channels) and "Reduce and Reuse" pruning (to cut output channels).
  The assessment of input channels is performed by applying the network to a sample of images and measuring the variance in a feature map. These measures are ranked and those below a threshold are pruned.
  The "Reduce and Reuse" method targets a reduction in the number of output channels produced by a convolutional layer. The underlying assumption is that output channels exhibiting low variation are less informative and can therefore be removed with minimal impact on performance. A key challenge of pruning output channels is that each removed channel is expected as an input by subsequent layers. To address this issue, the Reduce and Reuse method approximates the removed channels as linear combinations of the remaining ones. This approximation is obtained by solving a least-squares optimization problem that minimizes the reconstruction error between the original and approximated outputs. The resulting transformation is implemented as an additional 1×1 convolutional layer, allowing the network to preserve representational capacity while reducing the number of channels.
  The results from their experiments show that the variance based method is more effective than use of random pruning.

- **Data independent pruning methods**. Relay on the assumption that properties of the filters and output channels, as the proportion of zeros, influence the structure's importance.
  Hu et al. (2016) [11] propose APoZ-based network trimming, a pruning strategy that does not rely on data-driven measures such as feature map variance or entropy. Instead, their method is based on the observation that a large number of zero activations in an output feature map indicates redundancy. Specifically, they introduce the Average Percentage of Zeros (APoZ) as a metric to quantify the weakness of a filter, with higher APoZ values suggesting lower importance. After each pruning step, the network is fine-tuned, and the authors report that fine-tuning yields better results than retraining the network from scratch. When evaluating the proposed

method on VGG-16, the authors achieved a compression rate of 2.59 after six pruning iterations, while 2-3% higher Top-1/Top-5 accuracy than the original VGG-16 model.

- **Optimization based channel approximation pruning methods** propose to remove some channels and then re-optimize the remaining ones instead of deciding which channels are important, so that the layer behaves the same.
  One representative approach is FilterSketch, proposed by Lin et al. (2021) in [18]. Experimental results demonstrate that FilterSketch consistently outperforms first-order magnitude-based pruning methods, FilterSketch removes around 41.5% of the FLOPs and parameters while keeping the top-1 accuracy at 93.19%. Compared to 93.26% by the pre-trained model, the accuracy drop is negligible on CIFAR-10.

2.8 Apache TVM

Apache TVM is an open-source machine learning compiler stack designed to transform high-level model descriptions into efficient executable code for diverse hardware targets, including CPUs, GPUs, and specialized accelerators. In contrast to framework-centric execution, where a model is typically run as a sequence of library calls inside a general-purpose runtime, TVM adopts a compiler-first approach: the model is imported, represented in intermediate forms, transformed through a series of optimizations, lowered into tensor programs, and finally compiled into target-specific code and runtime artifacts. The central motivation behind TVM is performance portability, that is, the ability to obtain efficient inference across heterogeneous hardware without having to manually rewrite kernels for each platform [2].

Deep learning frameworks such as PyTorch and TensorFlow make models easy to define, train, and export, but they do not necessarily produce binaries that are well optimized for every deployment target. Their default execution paths often depend on pre-existing backend libraries and general-purpose runtimes, which may leave performance on the table, especially on edge devices or less common architectures. TVM aims to close this gap by treating model deployment as a compiler problem. Instead of only selecting from a fixed set of vendor-provided kernels, it can transform the graph, generate low-level tensor code, and tune the generated implementation for the target hardware.

From a systems perspective, TVM can be understood as a multi-stage compilation pipeline:

1. **Frontend import.** Models are imported from formats such as ONNX, TensorFlow, or framework-exported graphs and translated into TVM's internal representation.

2. **Graph-level optimization.** High-level transformations are applied, such as constant folding, dead-code elimination, operator fusion, and layout rewriting.

3. **Lowering to tensor programs.** Individual operators are lowered into explicit loop- and tensor-based computations that expose opportunities for schedule optimization.

4. **Schedule optimization and tuning.** The compiler explores implementation choices such as tiling, loop reordering, vectorization, unrolling, parallelization, memory placement, and thread binding.

5. **Code generation.** The optimized tensor programs are translated into target-specific code, for example via LLVM for CPUs or CUDA for NVIDIA GPUs.

6. **Runtime packaging and execution.** The compiled artifact is packaged together with a runtime component, such as a graph executor, virtual machine, or ahead-of-time runtime, depending on the deployment setting.

The key idea behind TVM is that performance optimization should occur at multiple abstraction levels. At the graph level, TVM can simplify and restructure the model to reduce overhead and memory traffic. At the operator level, it can search for efficient implementations of the remaining kernels. This is especially important because deep learning performance is often determined not only by arithmetic cost, but also by memory access patterns, tensor layout, synchronization overhead, and the degree to which the generated kernels match the microarchitectural properties of the target hardware. In this sense, TVM differs from a pure runtime system: it does not simply execute a graph, but attempts to synthesize an efficient implementation of that graph for a specific target.

A major contribution of TVM is the separation between the computation and the schedule. The computation describes what is to be computed, while the schedule describes how the computation is mapped to hardware. For example, the same convolution can be implemented with different tile sizes, loop orders, vector widths, thread mappings, and memory staging strategies. By exposing these choices explicitly, TVM enables systematic optimization instead of relying solely on fixed library kernels. This design is particularly valuable on hardware where vendor libraries are unavailable, suboptimal, or narrowly tuned for a limited set of workloads.

Another important feature of TVM is automated tuning. Rather than requiring all schedules to be written manually, TVM can search over a space of candidate implementations and evaluate them on the target device. The goal is to discover schedules that best exploit the target architecture's compute resources, cache hierarchy, memory bandwidth, and parallel execution model. In practice, this means that TVM can adapt the same model to very different platforms, ranging from embedded ARM CPUs to NVIDIA GPUs and more specialized accelerators. This tuning-based approach is one of the main reasons TVM is attractive for research and deployment on heterogeneous edge hardware.

TVM is particularly appealing in scenarios where model deployment must go beyond standard inference APIs. For example, it allows the developer to inspect intermediate representations, influence optimization passes, control scheduling decisions, and integrate custom compilation flows. This makes it useful not only as a deployment framework, but also as a research platform for studying compiler effects on machine learning inference. In the context of edge AI, where latency, memory traffic, and hardware utilization are critical, this level of control can be a major advantage.

At the same time, TVM also has several limitations. First, achieving strong performance often depends on the quality of the tuning process. An untuned TVM build may perform only modestly better than a generic runtime, and in some cases may perform worse. Second, the compilation workflow is more complex than that of more deployment-oriented frameworks such as ONNX Runtime or TensorRT. This complexity increases the engineering effort required to build a stable pipeline, especially when dealing with dynamic shapes, unsupported operators, or custom layers. Third, while TVM offers broad flexibility, this flexibility comes with a steeper learning curve: understanding its IR structure, scheduling model, and tuning workflow typically requires considerably more compiler knowledge than using a runtime-oriented backend. These trade-offs make TVM powerful, but not always the most practical choice for rapid deployment.

Compared with TensorRT, TVM is generally more open and more flexible, but often less turnkey. TensorRT is highly optimized for NVIDIA GPUs and provides aggressive graph- and kernel-level optimizations with relatively little manual effort, making it very effective for production inference on supported NVIDIA platforms. However, TensorRT is tied to the NVIDIA ecosystem and offers less control over compiler internals. TVM, by contrast, can target a broader range of devices and allows deeper intervention into compilation and scheduling, which is advantageous for research and for non-standard hardware targets. Compared with ONNX Runtime, TVM typically provides more opportunities for hardware-specific code generation and tuning, whereas ONNX Runtime emphasizes broad operator support, ease of use, and stable execution through execution providers. Thus, TVM occupies a distinctive position: it is best viewed not merely as an inference runtime, but as a programmable optimization stack for machine learning deployment.

Overall, the main strength of TVM lies in its ability to bridge the gap between high-level model representation and low-level hardware-aware optimization. Its compiler-driven design makes it particularly relevant when standard backends do not fully exploit the target platform, or when the goal is to study how graph transformations, scheduling, and code generation influence end-to-end inference performance. For this reason, TVM is a compelling framework in research on compiler-aware optimization of deep learning models for edge devices.

One of the central graph optimizations performed by TVM is operator fusion. Instead of executing each operation independently, multiple adjacent operations can be merged into a single fused region. This reduces kernel launch overhead, avoids writing intermediate tensors to memory, and may improve cache reuse.

A common example in neural networks is the fusion of convolution, bias addition, and activation:

$$y = \mathrm{ReLU}(\mathrm{Conv}(x, w) + b) \tag{2.1}$$

Without fusion, this sequence may be executed as three separate kernels:

1. convolution,
2. elementwise bias addition,
3. activation.

With fusion, the same sequence can be lowered into one kernel that computes the final output directly:

$$y_{n,c,h,w} = \max\left(0, \sum_{r,s,k} x_{n,k,h+r,w+s} \cdot w_{c,k,r,s} + b_c\right) \tag{2.2}$$

This eliminates the need to materialize the intermediate tensor produced by the convolution before activation.

A simplified illustration of the transformation is shown below:

```
Before fusion:
conv2d -> bias_add -> relu

After fusion:

fused_conv2d_bias_relu
```

In practice, TVM identifies fusible patterns during graph transformation passes and groups them into larger composite functions that are later lowered as a unit.

Tensor expression abstraction. At the operator level, TVM uses tensor-based abstractions to describe computation. In the original TVM design, a tensor expression specifies what should be computed, while a separate schedule specifies how it should be executed on hardware. This separation is fundamental because the same mathematical operation can be implemented efficiently in many different ways depending on the target architecture.

For example, matrix multiplication can be expressed as:

$$C[i,j] = \sum_{k} A[i,k] \cdot B[k,j] \tag{2.3}$$

A simplified TVM tensor-expression implementation is shown below.

```
from tvm import te
# Problem size
M, N, K = 1024, 1024, 1024
# Placeholders
A = te.placeholder((M, K), name="A")
B = te.placeholder((K, N), name="B")
# Reduction axis
k = te.reduce_axis((0, K), name="k")
# Compute definition
C = te.compute(
 (M, N),
lambda i, j: te.sum(A[i, k] * B[k, j], axis=k),
name="C"
)
```

This definition describes the mathematical computation only. It does not yet specify loop ordering, blocking, vectorization, or parallel execution.

Scheduling and kernel mapping. To obtain an efficient implementation, TVM applies scheduling decisions that determine how the tensor computation maps to loops, threads, and memory. Typical transformations include tiling, loop splitting, reordering, vectorization, unrolling, and thread binding. A simplified scheduling example for matrix multiplication on CPU is shown below:

```
s = te.create_schedule(C.op)
i, j = C.op.axis
```

```
k = C.op.reduce_axis[0]
# Tile output axes
io, ii = s[C].split(i, factor=32)
jo, ji = s[C].split(j, factor=32)
# Reorder loops
s[C].reorder(io, jo, k, ii, ji)
# Parallelize outer loops
s[C].parallel(io)
# Vectorize innermost loop
s[C].vectorize(ji)
```

Here, tiling improves locality, parallelization distributes work across CPU cores, and vectorization maps the innermost loop onto SIMD instructions. On GPUs, the same computation would instead be mapped to thread blocks and warps.

A simplified CUDA-style schedule might look as follows:

```
bx, tx = s[C].split(i, factor=16)
by, ty = s[C].split(j, factor=16)
s[C].bind(bx, te.thread_axis("blockIdx.x"))
s[C].bind(by, te.thread_axis("blockIdx.y"))
s[C].bind(tx, te.thread_axis("threadIdx.x"))
s[C].bind(ty, te.thread_axis("threadIdx.y"))
```

Such transformations demonstrate that TVM does not merely call a pre-existing kernel; rather, it can generate a hardware-aware implementation by explicitly controlling the execution structure.

Fusion and memory traffic reduction. The practical benefit of fusion can be understood through memory traffic. Suppose three operators are executed separately:

$$z = \mathrm{ReLU}(x + y) \tag{2.4}$$

If implemented naively as two kernels, the intermediate result $u = x + y$ must first be written to memory and then read again by the activation kernel. With fusion, the output is computed directly:

$$z_i = \max(0, x_i + y_i) \tag{2.5}$$

This reduces memory traffic from:

$$\text{read}(x) + \text{read}(y) + \text{write}(u) + \text{read}(u) + \text{write}(z) \tag{2.6}$$

to:

$$\text{read}(x) + \text{read}(y) + \text{write}(z) \tag{2.7}$$

For memory-bound workloads, such reductions can improve performance substantially, particularly on edge devices with limited memory bandwidth.

Auto-tuning and search. A major advantage of TVM is that these scheduling decisions need not be chosen manually. Instead, TVM can search over a space of candidate schedules and evaluate them on the target hardware. For example, the compiler may explore different tile sizes, vector widths, loop orders, or memory staging strategies in order to identify the implementation with the best measured latency.

A simplified view of this process is:

1. define the computation,
2. generate multiple candidate schedules,
3. benchmark each candidate on the target device,
4. select the best-performing schedule,
5. compile the final executable.

This search-based optimization is one of the reasons TVM is attractive for heterogeneous deployment: it can adapt the same model to different hardware backends without requiring a fully manual kernel engineering effort.

Relation to this thesis. In the context of this thesis, TVM is particularly relevant because it exposes the interaction between graph-level transformations and kernel-level execution efficiency. Unlike purely runtime-oriented frameworks, TVM allows direct control over lowering and scheduling decisions, which makes it suitable for studying how memory traffic, layout transformations, operator fusion, and target-specific code generation affect inference latency on edge platforms. This is especially valuable when analyzing whether bottlenecks arise from the model graph itself, from suboptimal kernel implementations, or from a mismatch between the deployed runtime and the target hardware.

2.9 TensorRT

2.10 Performance Bottlenecks in ML Inference

A workload is compute-bound if increasing the raw processing power (e.g., higher clock speed, more compute units) directly leads to a proportional decrease in execution time, assuming memory and latency are not limiting factors.

Inference for complex neural networks requires millions of multiply-accumulate (MAC) operations. If hardware is not fully utilized and code not map vectors/tensors effectively, the processing bottlenecks are created.

Many ML operations are memory-bound. This means the execution time is dominated by the time spent transferring data between memory levels (e.g., DRAM to caches, cache levels, host-to-device memory) rather than the computation itself.

Latency bounded operations: single-batch inference, overhear for dispatching operations, pipeline stalls.

These strategies reduce dispatch overhead for frequently executed execution paths:

- **Ahead-of-Time (AOT):** Compile the entire model before execution, as in traditional C/C++ workflows and many edge deployment scenarios.
- **Just-In-Time (JIT):** Compile code or computation graphs at runtime, typically upon first execution, leveraging runtime information such as tensor shapes, data types, and target hardware.

2.11 Relation to this thesis

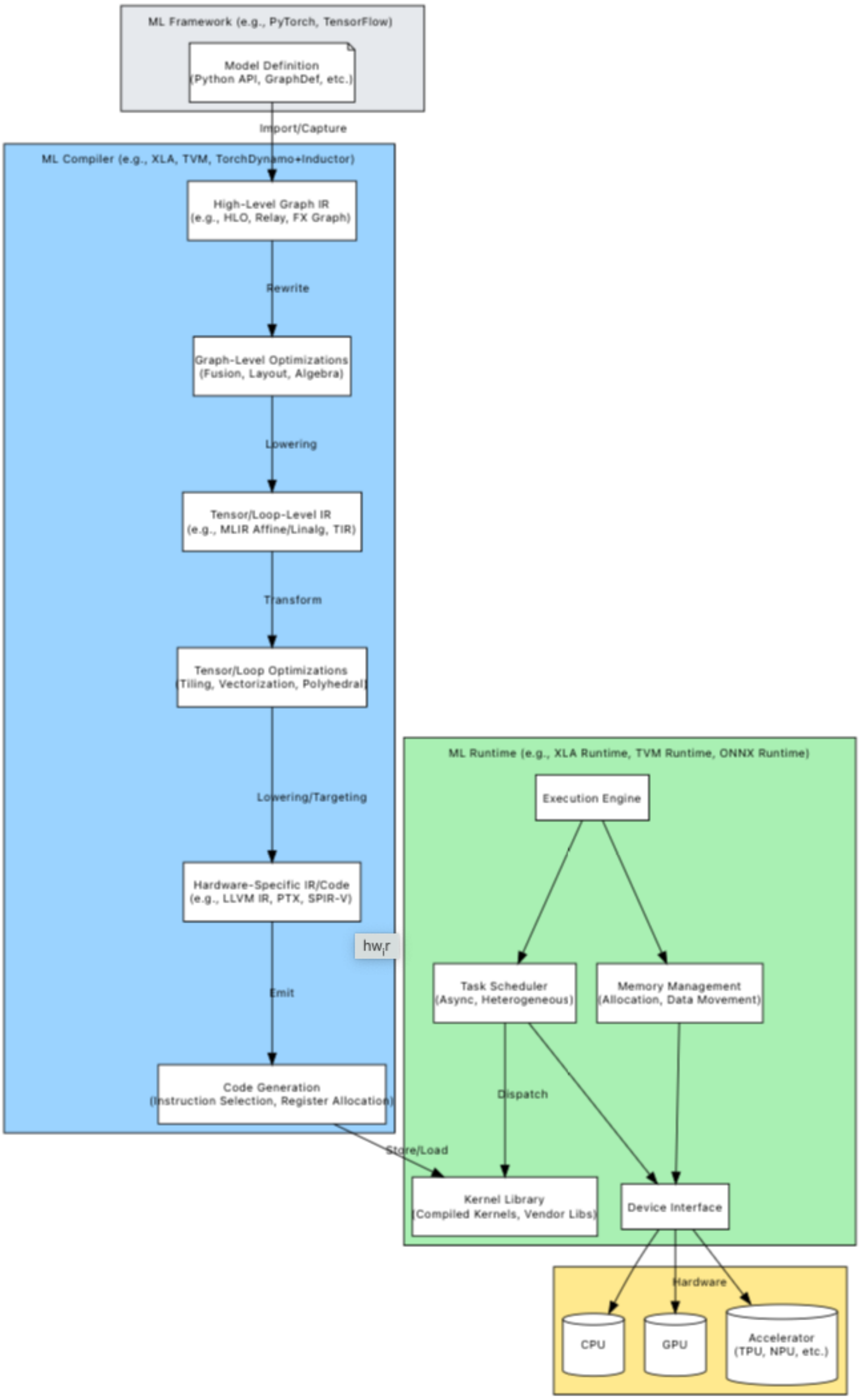

Figure 2.2: Flow of an ML model from definition through compilation and runtime execution on hardware.

3 Theory

3.1 Metrics Description

To evaluate the performance of object detection models, a set of standard metrics is used throughout this report. These metrics assess both localization accuracy and classification quality, providing a comprehensive view of model behavior.

- **Intersection over Union (IoU)**:
  For a predicted bounding box p and a ground-truth box g, the Intersection over Union is defined as:
  $$\mathrm{IoU}(p, g) = \frac{|p \cap g|}{|p \cup g|}.$$
  IoU measures localization accuracy by quantifying the overlap between predicted and ground-truth boxes. It is computed as the ratio of the intersection area to the union area of the two boxes, resulting in a value between 0 and 1. A score of 1 indicates perfect alignment, while 0 indicates no overlap.

  In practice, an *IoU threshold* (e.g., 0.5) is used to determine whether a prediction is correct. Predictions with IoU $\geq \tau$ are considered true positives (TP), while those below the threshold are treated as false positives (FP). IoU serves as a fundamental building block for higher-level evaluation metrics such as precision, recall, and mAP.

- **Precision and Recall**:
  Precision and recall evaluate the quality of classification and detection results. Precision measures the proportion of correct positive predictions:
  $$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}.$$
  Recall measures the proportion of correctly detected instances among all ground-truth objects:
  $$\mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}.$$
  Here, TP, FP, TN, and FN denote true positive, false positive, true negative, and false negative predictions, respectively. Precision reflects the ability to avoid false detections, while recall captures the ability to detect all relevant objects.

- **Average Precision (AP)**:
  Average Precision summarizes the precision-recall trade-off by computing the area

under the precision-recall (P–R) curve [5]:

$$\mathrm{AP}_\tau = \int_0^1 p(r;\tau)\,dr.$$

The P–R curve is constructed by varying the detection confidence threshold:

– Each detection is assigned a confidence score $s \in [0, 1]$.

– Detections are sorted in descending order of confidence.

– Predictions are progressively included, and precision and recall are recomputed.

– This process generates the precision-recall curve.

AP corresponds to the area under this curve and provides a single scalar measure of detection performance.

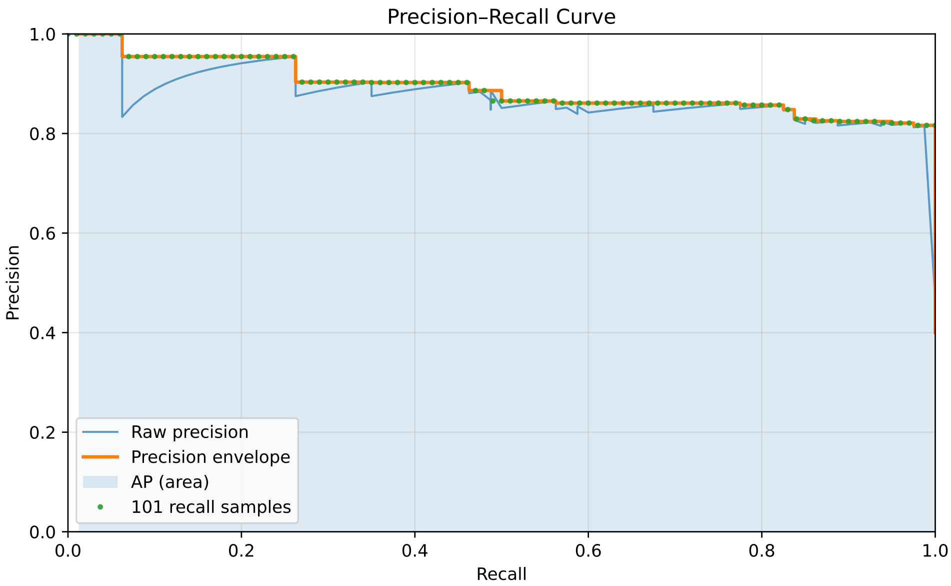

Figure 3.1: Precision–Recall curve.

- **Mean Average Precision (mAP)**:
  Mean Average Precision extends AP to multi-class settings by averaging AP over all object classes:

$$\mathrm{mAP}_{0.50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_{c,\,0.50},$$

$$\mathrm{mAP}_{[0.50:0.95]} = \frac{1}{C}\sum_{c=1}^{C} \frac{1}{10}\sum_{k=0}^{9} \mathrm{AP}_{c,\,(0.50+0.05k)}.$$

Here, C denotes the number of classes. The notation indicates the IoU threshold(s) used to determine true positives. For example, $\mathrm{mAP}_{0.50}$ considers predictions with

IoU ≥ 0.5, while mAP$_{[0.50:0.95]}$ averages performance across multiple thresholds, providing a more comprehensive evaluation.

3.2 Roofline Model

The Roofline model, introduced by Williams et al. [42], provides an intuitive way to analyze whether a workload is limited by memory bandwidth or computational throughput. It plots performance (e.g., GFLOP/s) versus arithmetic intensity (FLOPs per byte of memory traffic). The model defines two limits: a sloped memory roof determined by peak memory bandwidth, and a flat compute roof determined by peak floating-point throughput. Any kernel's performance is bounded by the minimum of these two ceilings. If a point lies on the sloped region, it is memory-bound; if it lies on the flat region, it is compute-bound. The Roofline model is widely used to analyze whether a workload is limited by memory movement or computation and to guide optimization efforts.

Optimization strategy of a neural network model is dependent on whether our device limits us in the speed of reading/writing to/from memory or the computational speed overall.

- **If the workload is Memory-Bound:**
  This means performance is limited by memory bandwidth, not by compute units. The arithmetic intensity (FLOPs/byte) is low, so the kernel spends most of its time moving data. The goal is then to either increase arithmetic intensity, or reduce memory traffic.
  Optimization strategies:
  - Improve data locality: reuse data in cashes (tiling/blocking); Keep tensors in shared memory (GPU);
  - Fuse operations to avoid writing intermediate tensors to memory (Conv $\rightarrow$ BatchNorm $\rightarrow$ Silu) $\rightarrow$ (Fused Conv-BN-Activation)
  - Reduce Precision (FP32 $\rightarrow$ FP16 $\rightarrow$ INT8 $\rightarrow$ INT4)
  - Reduce tensor size: pruning, structured sparsity, channel reduction, smaller input resolution
  - Improve memory layout: avoid unnecessary copies, channel-last format (NHWC)
- **If the workload is Compute-Bound:**
  This means arithmetic units are saturated. Arithmetic intensity is high enough that bandwidth is no longer limiting. The goal is then to increase compute efficiency. This case is harder, as it requires the work with scheduling, parallelization, kernel

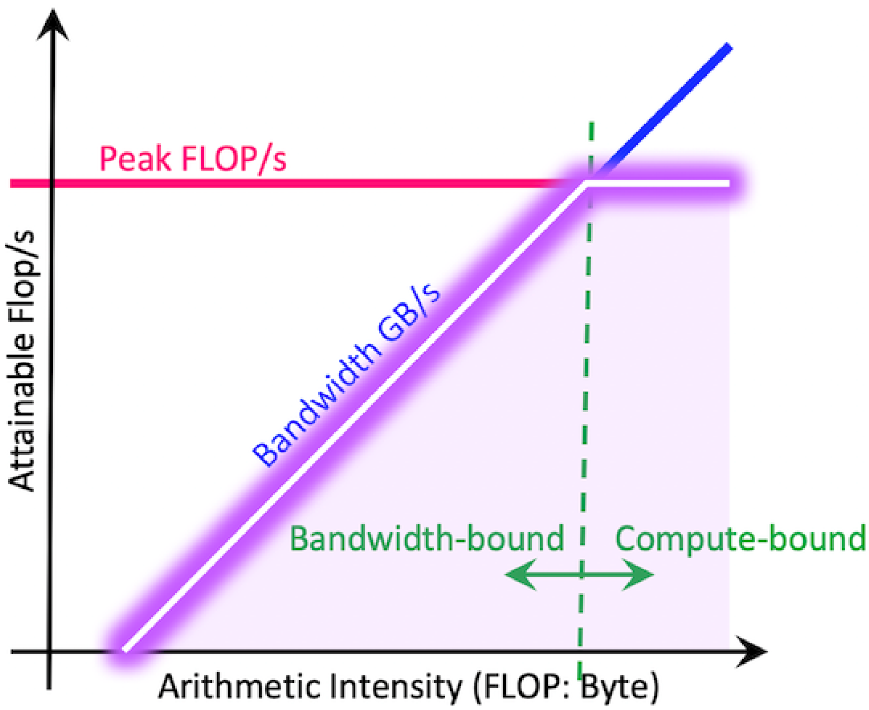

Figure 3.2: A Roofline model illustrating performance limits based on compute capability (horizontal line) and memory bandwidth (sloped line) versus the arithmetic intensity of operations (dots). Operations below the "roof" are limited by either memory bandwidth or compute peak.

optimization and algorithmic improvements.

- **Peak computational performance (in FLOP/s)** is obtained by benchmarking a compute-bound kernel that saturates the floating-point execution units. In practice, this can be achieved using large dense matrix multiplications of size $m \times k$ and $k \times n$, for which the floating-point operation count is given by:

$$\text{FLOPs} = 2 * m * k * n. \tag{3.1}$$

  The sustained peak performance is then computed as

$$P_{\text{peak}} = \frac{\text{FLOPs}}{t_{\text{compute}}}, \tag{3.2}$$

  where t_{compute} denotes the measured execution time.

- **Peak memory bandwidth (in GB/s)** is determined using a streaming memory benchmark. A large device-to-device copy or STREAM-like triad kernel is executed over a sufficiently large buffer to avoid cache effects. The sustained memory bandwidth is computed as

$$B_{\text{peak}} = \frac{\text{Bytes transferred}}{t_{\text{memory}}}. \tag{3.3}$$

- **Arithmetic intensity (AI)** is defined as the ratio between the number of floating-point operations and the volume of data transferred to and from main memory:

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes}}. \tag{3.4}$$

Since exact DRAM traffic is generally unavailable without hardware performance counters, a lower-bound estimate of memory traffic is commonly used. For convolutional or linear layers, this lower bound consists of the input tensor, weight parameters, and output tensor:

$$\text{Bytes}_{\text{LB}} = \text{Bytes}_{\text{input}} + \text{Bytes}_{\text{weights}} + \text{Bytes}_{\text{output}}. \tag{3.5}$$

This estimate ignores cache reuse and intermediate buffering, thereby providing an upper bound on arithmetic intensity.

- **The achieved performance of a workload** is computed as

$$P_{\text{achieved}} = \frac{\text{FLOPs}}{t_{\text{execution}}}. \tag{3.6}$$

The Roofline bound is then defined as

$$P_{\text{roof}}(\text{AI}) = \min\left(P_{\text{peak}},\ B_{\text{peak}} \times \text{AI}\right). \tag{3.7}$$

If P_{achieved} approaches $B_{\text{peak}} \times \text{AI}$, the workload is memory-bound, if it approaches P_{peak}, the workload is compute-bound. This methodology enables systematic classification of performance bottlenecks and guides optimization strategies such as improving data locality, increasing arithmetic intensity, or enhancing computational parallelism.

3.3 Activation Functions: ReLU and SiLU

Activation functions play a critical role in introducing non-linearity into neural networks, enabling them to learn complex representations. In this work, we consider two commonly used activation functions: Rectified Linear Unit (ReLU) and Sigmoid Linear

Unit (SiLU).

The ReLU activation function was introduced by [24] and is widely used due to its computational simplicity. More recently, smooth activation functions such as SiLU (or Swish) [34] have been proposed, offering improved accuracy at the cost of increased computational complexity.

The ReLU activation is defined as:

$$\mathrm{ReLU}(x) = \max(0, x) \tag{3.8}$$

ReLU is computationally simple, requiring only a comparison and selection operation. This makes it highly efficient on modern hardware, as it can be implemented with minimal arithmetic and memory overhead. Due to its simplicity, ReLU is widely used in performance-critical applications and is well-supported by hardware accelerators.

In contrast, the SiLU activation (also known as the Swish function) is defined as:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{3.9}$$

where $\sigma(x)$ denotes the sigmoid function. Compared to ReLU, SiLU introduces additional non-linearity and has been shown to improve model accuracy in many cases. However, it is computationally more expensive, as it requires evaluation of the exponential function, a division, and a multiplication.

From a computational perspective, the key difference between the two functions lies in their complexity. ReLU involves only a simple thresholding operation, which is typically memory-bound and incurs minimal computational cost. In contrast, SiLU requires multiple floating-point operations, including transcendental functions, making it more compute-intensive and potentially slower on hardware with limited support for such operations.

This difference has practical implications for model optimization. Replacing SiLU with ReLU can reduce inference latency and improve hardware efficiency, especially on edge devices. However, this simplification may lead to a degradation in model accuracy. Therefore, the choice of activation function represents a trade-off between computational efficiency and predictive performance.

While smooth activation functions such as SiLU have been shown to improve accuracy, recent work suggests that simpler functions like ReLU can offer advantages in terms of computational efficiency and activation sparsity [22]. This sparsity can be exploited by hardware and compilers to reduce the number of effective computations, leading to improved inference performance.

4 Methodology

This chapter describes the methodology used to evaluate and optimize deep learning inference on edge-oriented hardware platforms. The study is centered on the YOLOv8n object detection model and investigates how compiler-based optimization frameworks, numerical precision reduction, structural model simplification, and low-level kernel customization affect inference performance and detection accuracy.

The methodology is organized as a sequence of experimental stages. First, a baseline is established using the original FP32 PyTorch implementation of YOLOv8n. Next, bottlenecks are analyzed using hardware-aware performance models and profiling tools. Based on the observed limitations, several optimization directions are explored, including framework-level compilation, quantization, precision reduction, pruning, and custom TensorRT plugin development. Each optimization is evaluated with respect to latency, throughput, and accuracy in order to characterize the trade-offs between performance and model quality.

4.1 Model Selection

YOLOv8n is selected as the reference model for this study. The model represents a practical and relevant target for edge deployment because it combines a compact architecture with competitive detection accuracy. It is widely used in real-world object detection tasks, is actively supported by the research and engineering community, and can be exported to multiple deployment backends. These properties make it suitable for comparative analysis across inference frameworks.

From a methodological perspective, YOLOv8n is large enough to expose non-trivial optimization challenges, including operator fusion, memory movement overhead, kernel launch inefficiencies, and sensitivity to reduced precision. At the same time, it remains sufficiently lightweight to allow repeated experimentation, retraining, export, and validation on commonly available development hardware. This enables a controlled and reproducible study of both high-level and low-level optimization strategies.

4.2 Overall Experimental Procedure

The experimental workflow follows five main stages:

1. Establish a baseline using the original FP32 PyTorch model.
2. Identify performance bottlenecks through roofline analysis and runtime profiling.
3. Apply optimization techniques at the framework, numerical, and architectural levels.
4. Evaluate the optimized models on target hardware using identical benchmarking conditions.
5. Compare the resulting variants in terms of latency, throughput, and accuracy.

This procedure ensures that each optimization step is assessed relative to a common reference and that conclusions are based on measurable changes in both performance and predictive quality.

4.3 Baseline Establishment

The starting point of the study is the pretrained FP32 YOLOv8n model provided by Ultralytics and executed in the original PyTorch runtime. This baseline serves two purposes. First, it provides a reference for all later comparisons. Second, it makes it possible to determine whether the observed performance is already close to the limits of the target hardware or whether substantial optimization opportunities remain.

The baseline is evaluated on the selected hardware platforms under controlled conditions. For each platform, inference latency and throughput are measured using repeated runs with warm-up iterations to reduce initialization effects. Accuracy is evaluated on the validation dataset using standard object detection metrics. These baseline results form the reference against which all optimized variants are compared.

4.4 Bottleneck Analysis

After establishing the baseline, the next step is to determine the dominant performance limitations of the model on each platform. Two complementary approaches are used for this purpose: roofline-based reasoning and runtime profiling.

The roofline model is used as a coarse analytical framework to determine whether performance is likely to be limited by memory bandwidth or by available compute throughput. This provides a hardware-aware interpretation of the measured performance and helps identify whether the main limitation arises from insufficient arithmetic intensity, inefficient memory access, or underutilization of the compute units.

To refine this analysis, profiling tools are used to inspect the execution of individual operators and kernels. This makes it possible to identify layers with high latency, excessive memory movement, frequent kernel launches, or inefficient execution patterns. In particular, the profiling stage is used to answer the following questions:

- Which layers dominate end-to-end inference time?
- Whether the execution is primarily memory-bound or compute-bound.
- Whether the framework introduces avoidable overhead through layout conversions, intermediate buffers, or host-device synchronization.
- Whether operator fusion and backend-specific optimizations are effectively applied.

The outcome of this stage determines which optimization direction is most relevant for a given backend and hardware target.

4.5 Optimization Directions

Based on the bottleneck analysis, the study investigates several classes of optimization methods.

4.5.1 Compiler and Runtime Optimization

The first optimization direction consists of evaluating how different inference frameworks transform and execute the same model. Starting from the original PyTorch implementation, the model is exported or compiled for alternative backends such as TensorRT and TVM. The purpose of this comparison is to assess how much performance can be gained from compiler-level graph transformations, backend-specific kernel selection, operator fusion, and scheduling optimizations.

This stage is not limited to reporting end-to-end latency. It also examines how the internal execution differs between backends, for example in the number of kernels launched, the use of fused operators, the presence of data layout transformations, and the distribution of execution time across layers. In this way, the comparison provides insight not only into which backend is faster, but also into why it is faster.

4.5.2 Numerical Precision Reduction

The second optimization direction focuses on reducing numerical precision in order to decrease memory traffic and accelerate arithmetic operations. Two forms of precision reduction are considered in this work.

First, inference in FP16 is evaluated against the FP32 baseline. This comparison isolates the effect of reduced floating-point precision and shows whether the target hardware benefits from half-precision execution.

Second, quantization to INT8 is explored where supported by the backend. Quantization has the potential to reduce model size, memory bandwidth requirements, and arithmetic cost, but it may also introduce accuracy degradation. Therefore, each quantized model is evaluated not only in terms of runtime performance but also in terms of detection quality. The purpose of this stage is to determine whether the performance gains justify the loss, if any, in predictive accuracy.

4.5.3 Model Compression and Structural Simplification

The third optimization direction targets the structure of the model itself. Rather than relying only on framework-level improvements, this part of the study investigates whether the network can be simplified in ways that improve runtime characteristics.

Two approaches are considered. The first is pruning, which reduces model complexity by removing less important parameters or channels. The second is replacement of computationally expensive operations with simpler alternatives. In particular, the effect of replacing SiLU activations with ReLU is evaluated. This substitution is motivated by the fact that simpler activations may improve fusion opportunities and reduce kernel complexity, even if they slightly affect model accuracy.

The purpose of this stage is to understand whether modest architectural simplifications can produce a favorable trade-off between accuracy and efficiency, especially on hardware with limited compute capability or reduced support for complex fused kernels.

4.5.4 Low-Level TensorRT Customization

The final optimization direction extends beyond standard framework usage and examines low-level customization within TensorRT. In this stage, custom plugins are developed to replace selected subgraphs with tailored implementations. The main case considered in this work is a custom plugin for the C2f block.

The objective of this step is to investigate whether performance can be improved further by reducing backend overhead, controlling kernel behavior more explicitly, or eliminating inefficient execution patterns that are not resolved by the default TensorRT conversion pipeline. This stage complements the framework-level comparison by showing the extent to which manual low-level intervention can outperform standard automated optimization.

4.6 Evaluation Criteria

All model variants are evaluated using the same set of criteria in order to ensure consistent comparison.

The primary performance metrics are inference latency and throughput. Latency is used to measure the response time of a single inference pass, while throughput indicates the sustained processing rate under repeated execution. Where relevant, profiling data is also used to analyze kernel-level execution time and the contribution of individual layers to the total runtime.

The primary quality metric is object detection accuracy on the validation dataset. Standard detection metrics are used to verify whether a given optimization preserves acceptable predictive performance. This is particularly important for quantization, pruning, and activation replacement, where improvements in runtime may come at the cost of degraded detection quality.

The final analysis therefore considers both efficiency and accuracy, with emphasis on the trade-off between them. An optimization is not treated as beneficial solely because it accelerates execution; it must also maintain sufficient model quality for practical deployment.

4.7 Methodological Summary

The methodology combines baseline benchmarking, bottleneck analysis, framework comparison, numerical optimization, structural simplification, and low-level backend customization. The goal is not only to identify the fastest deployment configuration for YOLOv8n, but also to understand which classes of optimization are most effective on edge-oriented hardware and under which conditions they provide measurable benefit.

Figure 4.1 summarizes the scope of the study and the main experimental directions considered in this thesis.

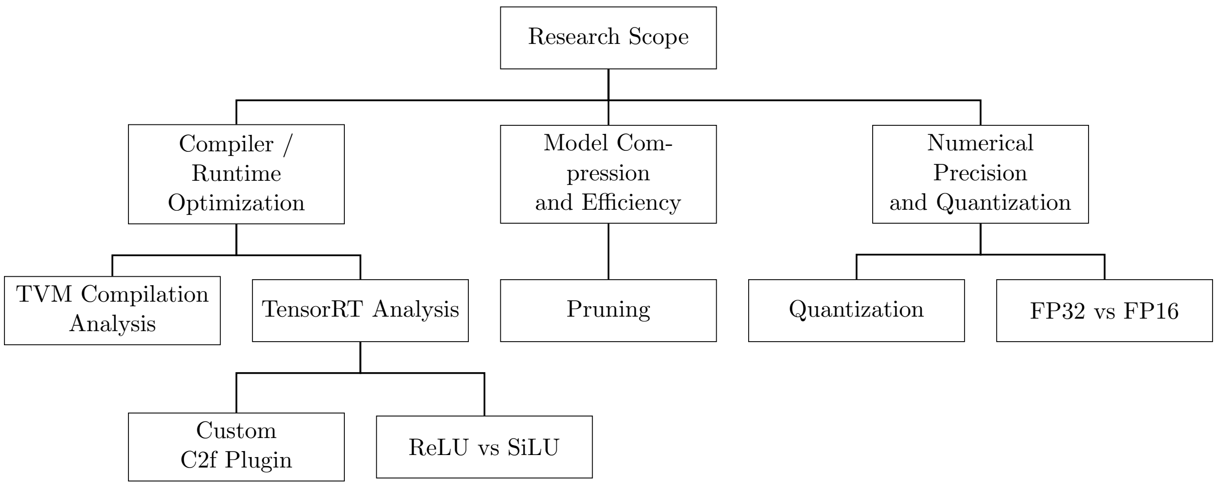

Figure 4.1: Overview of the thesis scope and main experimental directions.

5 Testing and Results

5.1 Experimental Setup

The experimental evaluation is conducted on two edge-oriented target platforms: the NVIDIA Jetson Orin Nano and the Raspberry Pi 5. These devices represent two different deployment scenarios for embedded inference: a GPU-accelerated AI platform with dedicated support for mixed-precision computation, and a general-purpose low-power ARM platform without a dedicated on-chip neural network accelerator.

Model training, export, and selected modification experiments, such as pruning and operator substitution, are performed on a host workstation equipped with an NVIDIA GeForce RTX 3070 GPU. This system is used as the primary development and experimentation environment, while all deployment-oriented inference benchmarks are executed on the target edge devices.

The Jetson Orin Nano is designed for edge AI and robotics workloads. Its system-on-chip integrates a 6-core Arm CPU, an Ampere-based GPU, and shared LPDDR5 memory accessible by both CPU and GPU. This architecture is particularly relevant for this study because it supports CUDA execution, Tensor Cores, mixed-precision inference, and INT8 acceleration, making it suitable for evaluating framework-level and low-level optimization strategies.

In contrast, the Raspberry Pi 5 represents a resource-constrained edge platform without a comparable integrated AI accelerator. It is therefore used to assess how optimization techniques behave on a more conventional embedded CPU-based system, where memory bandwidth and general-purpose compute resources are more limited.

Only the hardware characteristics most relevant to performance analysis are reported in this section, namely compute capability, memory capacity, and memory bandwidth.

Table 5.1: Host GPU specifications used for training and model development

Property	**Value**
Device	NVIDIA GeForce RTX 3070
Architecture	Ampere
FP32 Performance	∼20 TFLOPs
Memory	8 GB GDDR6
Memory Bandwidth	∼448 GB/s

Table 5.2: Specifications of the target edge devices

Property	**Jetson Orin Nano**	**Raspberry Pi 5**
CPU	6-core Arm Cortex-A78AE	4-core Arm Cortex-A76
GPU	Ampere (1024 CUDA cores)	VideoCore VII
FP32 Performance	∼0.5–1 TFLOPs (GPU)	—
Memory	8 GB LPDDR5	8 GB LPDDR4X
Memory Bandwidth	∼68 GB/s	∼25 GB/s
Accelerators	Tensor Cores	None*

* The Raspberry Pi 5 can be extended with external AI accelerators, such as the Hailo-8, via PCIe. However, such accelerators are not used in the experiments reported in this thesis.

5.2 Roofline Model

Constructing an accurate Roofline model for real-world workloads presents several challenges. First, the performance characteristics reported by hardware manufacturers often differ from measured values under practical conditions. Second, the effective behavior of a model depends on the execution framework, including factors such as cache utilization, memory reuse, and implicit optimizations (e.g., operator fusion), which are typically not visible at the model level.

Official YOLOv8 model characteristics are summarized in Table 5.3. The smallest variant, YOLOv8n, contains 3.2M parameters and requires 8.7B floating-point operations for a single forward pass at 640×640 resolution.

While vendor specifications indicate that the Raspberry Pi 5 provides up to 17 GB/s of memory bandwidth [36], and the Jetson Orin Nano offers up to 102 GB/s [25], measured performance is often significantly lower in practice.

Table 5.3: Comparison of YOLO models (trained on COCO) at 640×640 resolution [39].

Model	CPU ONNX (ms)	A100 TensorRT (ms)	Params (M)	FLOPs (B)
v8n	80.4	0.99	3.2	8.7
v8s	128.4	1.20	11.2	28.6
v8m	234.7	1.83	25.9	78.9
v8l	375.2	2.39	43.7	165.2
v8x	479.1	3.53	68.2	257.8

5.2.1 Roofline Analysis on Raspberry Pi 5

Using the methodology described in Chapter 3, the sustained memory bandwidth of the Raspberry Pi 5 was measured at 5.67 GB/s, and peak compute throughput at 73.95 GFLOP/s. All measurements were obtained with PyTorch configured to use 4 CPU threads, and layer timings were reported as the median over 20 runs after warmup.

These values define the device Roofline:

$$P(I) = \min\left(73.95,\ 5.67 \cdot I\right),$$

with a ridge point at approximately 13.04 FLOPs/byte, separating memory-bound and compute-bound regimes.

Table 5.4 summarizes representative per-layer performance characteristics of YOLOv8n executed on the Raspberry Pi 5 CPU using eager-mode PyTorch.

Layers with relatively low arithmetic intensity (AI $\approx$ 7–10 FLOPs/byte) achieve only modest throughput, typically in the range of 4.8–8.0 GFLOP/s, indicating memory-bound or memory-sensitive behavior under the measured Roofline parameters. In contrast, deeper convolutional layers with higher arithmetic intensity (AI $>$ 70 FLOPs/byte) achieve substantially higher effective throughput, often in the range of 40–60 GFLOP/s. Some layers with very high arithmetic intensity exceed 50 GFLOP/s, showing that smaller deep feature maps benefit from improved locality and higher compute utilization.

The distribution focal loss (DFL) convolution in the detection head has extremely low arithmetic intensity (0.47 FLOPs/byte) and achieves only 0.6 GFLOP/s, confirming strong memory limitation.

Although several individual convolutional layers exhibit relatively high effective throughput, the overall model achieves only 13.54 GFLOP/s, corresponding to approximately 18.3% of the measured device-level peak compute throughput of 73.95 GFLOP/s.

This discrepancy indicates that end-to-end inference performance is not determined solely by the efficiency of individual convolution kernels. Instead, it is also affected by

Table 5.4: Per-layer performance statistics of YOLOv8n on Raspberry Pi 5 (CPU, FP32).

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
0.conv	18.39	7.71	4.8	Memory-bound
2.cv1.conv	8.84	7.99	5.9	Memory-bound
2.cv2.conv	9.84	9.59	8.0	Memory-bound
4.m.0.cv2.conv	2.74	70.42	43.1	Compute-favored
6.m.1.cv2.conv	1.93	122.03	61.1	Compute-favored
18.m.0.cv2.conv	2.02	122.03	58.5	Compute-favored
22.cv3.0.1.conv	13.53	170.41	54.5	Compute-favored
22.dfl.conv	1.89	0.47	0.6	Strongly memory-bound
Full Model	327.00	157.76	13.54	Mixed / overhead-dominated

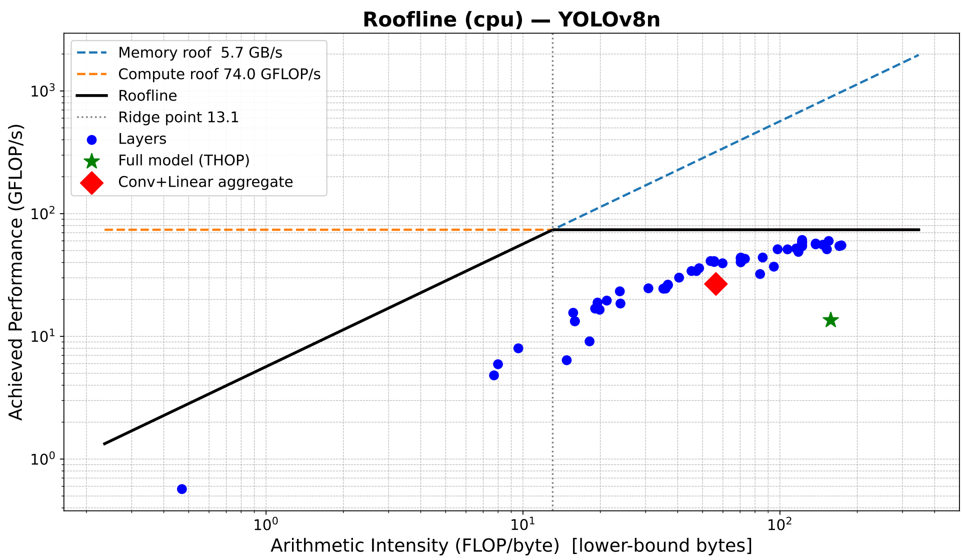

Figure 5.1: Roofline model of YOLOv8n on Raspberry Pi 5 (CPU, FP32).

system-level and framework-level overheads, including kernel dispatch overhead, synchronization, non-convolutional operations, tensor reshaping and concatenation, inter-layer memory traffic, and limited fusion between adjacent operators. Thus, the Roofline serves primarily as an upper bound and an interpretive tool rather than a directly attainable end-to-end target.

From an optimization perspective, these results suggest that further improvements should focus not only on reducing memory traffic, but also on improving execution efficiency at the graph and kernel level. Promising directions include operator fusion, graph-level optimization, backend-specific kernel tuning, quantization, and reducing framework overhead through more optimized runtimes.

The observed performance behavior can be further explained by the relationship between layer activation sizes and the cache hierarchy:

- L1: 64 KB per core
- L2: 512 KB per core
- L3: 2 MB shared
- DRAM bandwidth: $\approx$ 5.67 GB/s (measured)

In particular, the shared L3 cache of the Raspberry Pi 5 is limited to 2 MB, which constrains the amount of feature-map data that can be reused without returning to main memory.

Early layers of YOLOv8n operate on high-resolution feature maps (e.g., 320×320), resulting in activation tensors that far exceed the available cache capacity. Consequently, these layers experience frequent DRAM accesses and comparatively low effective performance, which is consistent with their lower arithmetic intensity and more memory-bound behavior.

In contrast, deeper layers operate on progressively smaller feature maps (e.g., 40×40 or 20×20), allowing a larger fraction of their working set to remain in cache. This improves locality and data reuse, enabling substantially higher effective throughput. Thus, even though the full model remains far below the ideal Roofline bound, the layerwise results still reveal a clear transition from memory-bound early layers to more compute-favored deeper layers.

5.2.2 Roofline Analysis on Jetson Orin Nano

The same analysis was performed on the NVIDIA Jetson Orin Nano GPU. With PyTorch configured to use a single host thread and using median timings after warmup, the measured sustained memory bandwidth was 55.68 GB/s, while peak compute throughput reached 810.68 GFLOP/s.

The resulting Roofline model is defined as:

$$P(I) = \min\left(810.68,\ 55.68 \cdot I\right),$$

with a ridge point at:

$$I^* = \frac{810.68}{55.68} \approx 14.56 \text{ FLOPs/byte.}$$

Compared to the Raspberry Pi 5, the Orin Nano exhibits both substantially higher compute throughput and much higher memory bandwidth. At the same time, the ridge point remains in a moderate arithmetic-intensity range, which means that many YOLOv8n convolution layers lie to the right of the knee and can benefit from the GPU's high compute capability.

Table 5.5: Representative per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32).

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	0.78	7.71	114.1	Memory-bound
model.2.cv1.conv	0.26	7.99	204.2	Memory-bound
model.4.m.0.cv2.conv	0.56	70.42	211.8	Compute-bound
model.6.m.1.cv2.conv	0.23	122.03	502.4	Compute-bound
model.12.m.0.cv2.conv	0.24	122.03	499.3	Compute-bound
model.22.cv2.0.1.conv	0.56	137.80	842.6	Compute-bound
model.22.cv3.0.1.conv	0.83	170.41	892.5	Compute-bound
model.22.dfl.conv	0.27	0.47	3.9	Strongly memory-bound
Full Model	26.02	157.76	170.22	Mixed / Overhead-dominated

The full model achieves 170.22 GFLOP/s, corresponding to approximately 21.0% of the measured peak compute throughput. This is substantially higher than on the Raspberry Pi 5, but still far below the theoretical device peak.

Per-layer analysis shows that many convolutional layers achieve several hundred GFLOP/s, with some layers approaching or even exceeding 800 GFLOP/s. In particular, layers with high arithmetic intensity (AI $>$ 70 FLOPs/byte) clearly operate in the compute-bound regime and make effective use of the GPU's compute resources.

However, low-intensity layers remain limited by memory behavior. The DFL convolution in the detection head, with AI $\approx$ 0.47 FLOPs/byte, achieves only 3.9 GFLOP/s and remains strongly memory-bound even on this more capable platform.

Table 5.5 highlights that, compared to the Raspberry Pi 5, the majority of convolutional layers on the Orin Nano operate in the compute-bound regime and achieve

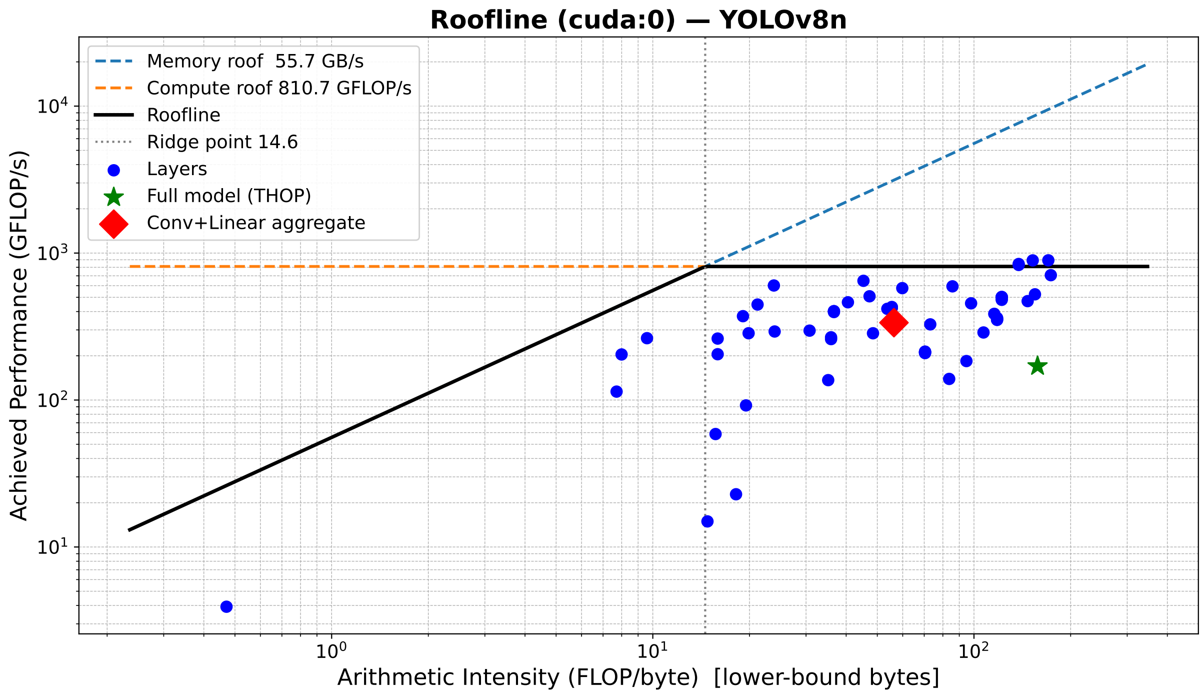

Figure 5.2: Roofline model of YOLOv8n on Jetson Orin Nano (GPU, FP32).

substantially higher throughput due to the GPU's much larger compute capacity and memory bandwidth.

At the same time, the results also show that high per-layer kernel efficiency does not directly translate into equally high end-to-end model efficiency. Although several layers approach the measured peak throughput, the overall model reaches only 170.22 GFLOP/s. This indicates the continued presence of system-level overhead, including kernel launch overhead, intermediate tensor movement, non-convolutional operations, and synchronization costs between layers.

Thus, the Orin Nano provides a much more favorable platform for YOLOv8n inference than the Raspberry Pi 5, but the Roofline analysis still reveals that end-to-end performance remains constrained by factors beyond raw compute throughput alone.

Detailed per-layer benchmarking results are provided in Appendix A.

5.3 Performance Comparison

We evaluated the performance of YOLOv8n models on two hardware platforms: the NVIDIA Orin Nano and the Raspberry Pi 5. Following this evaluation, we investigated different model compilation approaches tailored to each target platform.

Due to substantial differences in hardware architectures, the set of available inference frameworks also differs. Hardware-specific frameworks such as TensorRT are limited to

NVIDIA GPUs and therefore cannot be used for a direct cross-platform comparison. Instead, we assess the performance improvements provided by each framework relative to a common baseline on the same hardware.

The baseline execution is performed in PyTorch eager mode. In this mode, operations are executed immediately as Python code is interpreted line by line. The computational graph is constructed dynamically at runtime, which makes debugging straightforward and enables inspection of intermediate results, such as per-layer performance metrics presented in the previous section. However, eager execution introduces several limitations: Python-level overhead for each operation, lack of global graph-level optimization, and limited hardware-specific tuning. As a result, the model is effectively executed as a sequence of independent operators rather than as an optimized computational graph.

To address these limitations, the model can be treated as a static graph and optimized using compiler-based frameworks. A variety of such compilers have been developed to enable hardware-aware optimizations, including operator fusion, memory planning, and kernel selection. In this section, we evaluate these approaches on the target hardware platforms.

A standard step in this workflow is converting the model to the ONNX (Open Neural Network Exchange) format. In this work, we use ONNX export with opset version 17 and enable graph simplification. The opset defines the set of available operators (e.g., Conv, Add, Resize) and their semantics. Choosing an appropriate opset version is important: newer opsets may not be fully supported by all backends, while older versions may lack required operators or optimizations. The opset selection can therefore impact operator decomposition, kernel count, memory traffic, and opportunities for fusion.

Graph simplification is applied to reduce computational redundancy and streamline the graph structure. This includes:

- Constant folding: precomputing constant expressions (e.g., $x + (2 \times 3) \rightarrow x + 6$)
- Node elimination: removing redundant operations such as identity mappings
- Subgraph simplification: collapsing sequences of operations into simpler equivalents when possible
- Dead code elimination: removing unused branches of the graph

While these transformations simplify the graph, backend-specific compilers (e.g., TensorRT, TVM) may perform additional, more aggressive optimizations such as kernel fusion and layout transformations.

Once converted, the ONNX model can be executed using a variety of inference frameworks. On CPU-based platforms, commonly used frameworks include PyTorch (via Torch-

Script or TorchInductor), ONNX Runtime, and TensorFlow Lite. These frameworks provide different levels of optimization, ranging from general-purpose execution to lightweight deployment on resource-constrained devices.

On GPU-based platforms, available options include PyTorch with CUDA execution, ONNX Runtime with GPU or TensorRT execution providers, and TensorRT as a dedicated compiler and runtime for NVIDIA hardware. TensorRT performs aggressive graph-level and kernel-level optimizations, such as operator fusion, precision calibration (e.g., FP16/INT8), and memory layout transformations, making it particularly effective for high-performance inference on NVIDIA devices.

Another notable framework that supports both CPU and GPU targets is Apache TVM. Unlike runtime-focused frameworks, TVM operates as a deep learning compiler stack that performs end-to-end optimization, including graph transformations, operator scheduling, and hardware-specific code generation. This approach provides a high degree of flexibility and control over low-level execution compared to more API-driven frameworks, although it typically requires more manual tuning and expertise.

5.3.1 TVM MetaSchedule Tuning

Unlike the other evaluated deployment frameworks, TVM does not rely only on a fixed set of pre-implemented backend optimisations. Instead, it exposes the execution schedule itself as an optimisation problem and searches for hardware-specific implementations of individual operators. In practice, this means that TVM requires an additional tuning stage before final compilation, in which candidate schedules are generated, measured on the target device, and ranked according to their runtime performance. This makes TVM fundamentally different from frameworks such as ONNX Runtime, LiteRT, or PyTorch eager mode, which mainly depend on predefined kernels and runtime-level optimisations.

For this reason, the TVM results are discussed separately from the general framework comparison. The purpose of the following subsection is not only to report the final benchmark numbers, but also to analyse how MetaSchedule improved the execution of YOLOv8n, which operators benefited most from tuning, and where the main remaining bottlenecks are. This additional discussion is important because, in the case of TVM, the observed inference performance is directly shaped by the quality of the schedule search rather than by the default behaviour of a fixed runtime backend.

5.3.2 TVM MetaSchedule Tuning Results on Raspberry Pi 5 - FP32

TVM MetaSchedule was evaluated on the Raspberry Pi 5 (4-core Cortex-A76, AArch64, 2.4 GHz) using the full YOLOv8n inference graph imported through the Relax frontend. The tuning run lasted approximately 6 hours and 15 minutes, during which TVM explored 7 221 valid schedules across 65 extracted operator tasks. The resulting tuned model achieved a summed task latency of 173.6 ms, while the measured end-to-end inference latency was approximately 185–188 ms. The difference can be attributed to runtime overhead outside the tuned kernels, including Python execution, input preparation, and postprocessing in the detection head.

General Tuning Behaviour. The tuning results show that TVM was able to improve the execution of the convolution-heavy parts of the network through schedule search and cache-aware tiling. However, no task in this run could be tensorised into a dedicated ARM intrinsic pattern. In practice, this means that the observed performance gains came mainly from improved loop ordering, tiling, and LLVM-driven vectorisation, rather than from explicitly tensorised micro-kernel generation. This is an important observation because it highlights both the strength and the limitations of the current FP32 compilation path on Raspberry Pi 5: TVM can still produce competitive schedules, but it does not yet exploit the hardware as fully as a tensorised implementation would.

Best-Performing Operators. The highest efficiency was achieved by 1×1 convolutions, particularly in the neck and detection head. These layers are structurally simpler than 3×3 convolutions and map more naturally to vectorised execution, which allowed them to reach throughput above 80 GFLOPS in the best cases. Several mid-network 3×3 convolutions also performed well once the channel dimensions became sufficiently large. For such layers, the tuned schedules were able to keep the working set in cache and reduce memory traffic, which is especially important on a bandwidth-constrained CPU platform.

Main Bottlenecks. The weakest-performing operator was the initial stem convolution. Although its FLOP count is relatively modest, it operates on a very large spatial input and only three input channels, resulting in poor arithmetic intensity and limited opportunity for data reuse. Consequently, it consumed a disproportionately large share of inference time. Other early-stage 3×3 convolutions with low channel counts and large feature maps showed similar behaviour. In addition, the softmax operation in the detection head remained highly inefficient, not because of high computational complexity, but because it is dominated by repeated memory accesses and reductions rather than dense arithmetic.

Weighted Bottleneck Analysis. A per-layer view alone does not fully capture the true runtime bottlenecks of YOLOv8n. Some convolution tasks appear multiple times in the graph, especially inside repeated C2f blocks, and therefore contribute much more to total inference time than their individual latency would suggest. In the tuned model, several repeated 3×3 convolutions in the backbone and neck became the dominant contributors to total runtime once their multiplicity was taken into account. This confirms that tuning effort should be focused not only on the slowest individual operators, but also on operators that are executed many times.

Overall Assessment. Overall, the tuned TVM model reached an end-to-end latency of about 192 ms on Raspberry Pi 5, compared with approximately 290.1 ms for PyTorch eager mode under comparable conditions. The improvement appears to come mainly from better cache-aware scheduling of convolution layers, particularly the more expensive 3×3 operators in the middle of the network.

At the same time, the results also highlight the limitations of FP32 tuning on this platform. While MetaSchedule improved many layers, the absence of tensorisation meant that the performance gains remained incremental rather than transformative. The most difficult kernels were the early low-channel convolutions, where memory access patterns dominate and schedule search alone has limited effect. Therefore, the main conclusion is that TVM tuning is beneficial on Raspberry Pi 5, but the achievable gains in FP32 remain bounded. Larger improvements would likely require either a more targeted tuning budget for the most expensive tasks or a reduced-precision compilation path such as FP16.

5.3.3 TVM MetaSchedule Tuning Results on the Jetson Orin Nano GPU - FP32

TVM MetaSchedule was evaluated on the NVIDIA Jetson Orin Nano GPU using the full YOLOv8n inference graph imported through the Relax frontend. TVM explored 7 257 valid schedules across the same 65 extracted operator tasks. The resulting tuned model achieved a summed task latency of 22 920 μs, while the measured end-to-end inference latency was 23.38 ms. The difference between these values was small, indicating that the tuned kernel execution accounted for almost all of the observed runtime.

Compared with the Raspberry Pi 5 CPU result of 173 591 μs summed task latency, the GPU tuning result corresponds to a kernel-level speedup of approximately 7.6×. This confirms that the transition from CPU to GPU substantially accelerated the convolution-dominated parts of the model, even though the graph structure and extracted operator tasks remained unchanged.

General Tuning Behaviour. The tuning results show that TVM was able to improve execution on the GPU by searching over CUDA-specific schedule parameters such as thread-block structure, tiling configuration, and memory movement patterns. However, as in the CPU case, no task in this run could be tensorised into a dedicated intrinsic pattern. In practice, this means that the observed gains came from improved CUDA scheduling rather than from an explicit tensor-core-style mapping. This is an important observation, because it highlights both the strength and the limitation of the current FP32 GPU compilation path: TVM can still generate competitive CUDA schedules, but it does not yet exploit all hardware-specific acceleration paths that may be available on the platform.

Best-Performing Operators. The most efficient GPU kernels were large mid-network 3×3 convolutions, particularly those in repeated C2f blocks and in the neck. Unlike the CPU case, where 1×1 convolutions achieved the highest throughput, the GPU benefited most from larger dense convolutional operators with sufficiently high channel counts to expose substantial parallelism. The best-performing task reached 585.6 GFLOPS, while several other large 3×3 convolutions exceeded 500 GFLOPS. These results indicate that once the working set becomes large enough, the GPU can effectively amortise memory movement and keep a large number of execution units occupied.

Main Bottlenecks. The weakest-performing convolution remained the initial stem layer. Although it benefited substantially from GPU acceleration compared to the CPU, it still achieved much lower efficiency than the large mid-network operators. The reason is structural: the layer has only three input channels and a very large spatial resolution, which limits arithmetic intensity and reduces the amount of useful parallel work available per memory access. Several other early-stage 3×3 convolutions with low channel counts showed similar behaviour.

The softmax operation in the detection head also remained inefficient. Its absolute latency was much lower than on the CPU, but its achieved throughput was still very small compared with dense convolutions. This indicates that the operation remained dominated by reductions and repeated memory access rather than by highly parallel arithmetic.

Weighted Bottleneck Analysis. As in the CPU analysis, a per-layer view alone does not fully capture the true runtime bottlenecks of YOLOv8n. Several convolution tasks appear multiple times in the graph, especially inside repeated C2f blocks, and therefore contribute more to total inference time than their individual latency would suggest. On the GPU, the dominant weighted bottlenecks were again repeated 3×3 convolutions in the backbone and neck. In particular, task 50 (`conv2d24`) contributed the largest weighted latency, followed by task 16 (`conv2d11`), which appeared seven times in the graph. This is qualitatively similar to the CPU case: the most important optimisation targets are not necessarily the single slowest kernels, but rather the kernels that are executed many times.

Overall Assessment. Overall, the tuned TVM model reached an end-to-end latency of 23.38 ms on the Jetson Orin Nano GPU, compared with approximately 25.12 ms for PyTorch eager mode under comparable conditions. This means that TVM provided a measurable but modest improvement over the baseline GPU execution path. At the same time, the results also highlight the remaining limitations of the current FP32 TVM GPU path. While MetaSchedule reduced the latency of the convolution-heavy parts of the model substantially relative to the CPU, the absence of tensorisation meant that the gains remained bounded relative to the best available inference engine. In this experiment, TensorRT remained significantly faster at 11.38 ms, indicating that additional performance is still available beyond the schedules found by MetaSchedule. Therefore, the main conclusion is that TVM tuning is effective on the Jetson Orin Nano GPU and produces a strong result, but in FP32 it remains a competitive general solution rather than the fastest deployment path for this workload.

5.3.4 Raspberry Pi 5 Benchmark Overview - FP32

All Raspberry Pi 5 experiments were conducted on a Linux system based on the `6.12.62+rpt-rpi-2712` kernel using the AArch64 architecture. The CPU frequency governor was fixed to `performance`, and the CPU was configured to 2.4 GHz during the measurements. To reduce thermal and scheduling-related variation, the benchmarking procedure included a cooldown phase before each repetition and checks for frequency throttling. Each backend was evaluated over five repeated runs, with 200 timed inferences per repetition after 30 warm-up iterations. In addition, each repetition was executed in a fresh subprocess in order to reduce cross-framework interference, such as cache contamination or residual runtime state. The comparison was performed using all four available CPU cores on the Raspberry Pi 5 in order to reflect the maximum CPU throughput available in this deployment setting. This setup was chosen to reduce short-term variance and to provide a stable estimate of sustained inference performance under controlled CPU conditions.

The evaluated CPU backends were TVM, ONNX Runtime, TorchCompile, Google AI Edge LiteRT, and PyTorch eager mode used as a baseline. ONNX Runtime was evaluated with the `CPUExecutionProvider`. Table 5.6 summarises the measured latency and throughput, while Table 5.7 summarises power results.

Table 5.6: End-to-end YOLOv8n inference performance on Raspberry Pi 5 CPU. Values are reported as mean latency ± half-width of the 95 % confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
TVM	192.0 ± 7.5	5.21	188.8
ONNX Runtime	189.2 ± 7.0	5.28	187.2
TorchCompile	202.9 ± 8.5	4.93	202.2
LiteRT	154.1 ± 2.5	6.49	155.2
PyTorch eager	290.1 ± 2.3	3.45	289.8

Overall Performance. Among all evaluated frameworks, LiteRT achieved the best overall latency on the Raspberry Pi 5, with a mean inference time of 154.1 ms and a throughput of 6.49 FPS. ONNX Runtime and TVM formed the next performance group, with mean latencies of 189.2 ms and 192.0 ms respectively. TorchCompile was slower than both compiled deployment backends, reaching a mean latency of 202.9 ms and a throughput of 4.93 FPS. PyTorch eager mode remained the slowest configuration, with a mean latency of 290.1 ms and a throughput of 3.45 FPS. The difference between ONNX Runtime and TVM was only 2.8 ms, whereas TorchCompile lagged behind ONNX Runtime by 13.6 ms and behind TVM by 10.8 ms.

Table 5.7: Average power consumption during YOLOv8n inference on Raspberry Pi 5 CPU.

Backend	Mean Power (mW)	Max Power (mW)	Std (mW)
TVM	4096.2	4783.1	46.1
ONNX Runtime	4043.2	5415.1	172.5
TorchCompile	2291.8	2742.5	63.2
LiteRT	2292.8	2591.5	14.2
PyTorch eager	2316.4	2759.8	30.3

Stability of Measurements. The repeated-run statistics show that TVM and ONNX Runtime had comparable variability, with standard deviations of 6.03 ms and 5.65 ms respectively. TorchCompile showed a similar but slightly higher variability of 6.86 ms. In contrast, PyTorch eager mode and LiteRT were more stable, with standard deviations of 1.84 ms and 2.05 ms. LiteRT therefore combined the best average performance in this benchmark with comparatively consistent execution.

Power Behaviour. TVM and ONNX Runtime showed similar average power draw, measuring 4.10 W and 4.04 W respectively. In contrast, TorchCompile, LiteRT, and PyTorch eager mode all operated near 2.3 W average board power. LiteRT exhibited the lowest average power consumption at 2.29 W, although the difference relative to TorchCompile was very small. This result is notable because LiteRT was both the fastest backend in this experiment and the one with the lowest measured average power consumption.

Interpretation of the Results. Although TVM did not outperform LiteRT, it remained competitive with ONNX Runtime and outperformed TorchCompile in this end-to-end benchmark. TVM achieved slightly lower latency than TorchCompile while relying on a fully compiler-driven deployment pipeline based on Relax and MetaSchedule tuning. This suggests that ahead-of-time compilation through a dedicated inference compiler remains more effective on this workload than applying `torch.compile` to the original PyTorch model on CPU. At the same time, the results also show that LiteRT provided the most efficient execution path among the tested frameworks on Raspberry Pi 5 for this FP32 model.

Conclusion. The benchmark results show four practical performance tiers on the Raspberry Pi 5 CPU. LiteRT delivered the best end-to-end result, combining the lowest latency, the highest throughput, and the lowest average power consumption. ONNX Runtime and TVM achieved similar performance, with ONNX Runtime being slightly faster in the measured configuration. TorchCompile improved substantially over PyTorch eager mode, but remained slower than both ONNX Runtime and TVM. PyTorch eager mode was the slowest baseline by a large margin.

Relative to the PyTorch eager baseline, TVM achieved a 1.51× speedup, corresponding to a 33.8 % reduction in mean inference latency. ONNX Runtime achieved a 1.53× speedup, corresponding to a 34.8 % latency reduction. TorchCompile achieved a 1.43× speedup, corresponding to a 30.1 % latency reduction. LiteRT provided the largest improvement, reaching a 1.88× speedup and a 46.9 % reduction in mean latency. Overall, these results indicate that TVM is capable of producing competitive CPU inference performance on Raspberry Pi 5, but in this experiment the most efficient backend was LiteRT rather than TVM.

5.3.5 Jetson Orin Nano GPU Benchmark Overview - FP32

GPU experiments were conducted on the NVIDIA Jetson Orin Nano using the full YOLOv8n model in FP32 precision with batch size 1. The evaluated backends were TVM, ONNX Runtime, TensorRT, PyTorch eager mode, and `torch.compile`. As in the CPU experiments, the benchmark procedure used isolated repetitions in order to reduce interference between backends. Table 5.8 summarises the measured latency and throughput, while Table 5.9 summarises power results.

Table 5.8: End-to-end YOLOv8n inference performance on the NVIDIA Jetson Orin Nano GPU. Values are reported as mean latency ± half-width of the 95 % confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
TVM	23.38 ± 0.02	42.77	23.39
ONNX Runtime	23.18 ± 0.04	43.14	23.17
TorchCompile	72.34 ± 0.47	13.82	72.26
TensorRT	11.38 ± 0.02	87.87	11.39
PyTorch eager	25.12 ± 0.27	39.81	25.12

Overall Performance. TensorRT achieved by far the best end-to-end latency on the Jetson Orin Nano, reaching 11.38 ms, which corresponds to 87.9 FPS. ONNX Runtime and TVM formed a second performance group with nearly identical results, at 23.18 ms and 23.38 ms respectively. PyTorch eager mode was slightly slower at 25.12 ms, while `torch.compile` performed substantially worse than all other GPU backends, reaching 72.34 ms.

Relative to the PyTorch eager baseline, TVM achieved a 1.07× speedup, corresponding to a 6.9 % reduction in mean latency. ONNX Runtime achieved a 1.08× speedup and a 7.7 % latency reduction. TensorRT provided the largest improvement, reaching a 2.21× speedup and a 54.7 % reduction in mean latency. In contrast, `torch.compile` was slower than PyTorch eager mode by a factor of 2.88×.

Table 5.9: Average power consumption during YOLOv8n inference on the NVIDIA Jetson Orin Nano GPU.

Backend	Mean Power (mW)	Max Power (mW)	Std (mW)
TVM	26007.9	26781.1	88.7
ONNX Runtime	26952.4	29632.6	84.8
TorchCompile	18599.2	19967.0	76.7
TensorRT	28405.8	30936.2	86.4
PyTorch eager	25884.4	29186.0	168.4

Power and Efficiency. In absolute power draw, the GPU backends operated within a relatively narrow range, from 18.6 W for `torch.compile` to 28.4 W for TensorRT. However, when throughput is taken into account, TensorRT was also the most power-efficient backend, achieving 3.10 FPS/W. This is approximately 1.9 times higher than TVM and ONNX Runtime, which both reached about 1.6 FPS/W, and about twice the efficiency of

PyTorch eager mode. Although `torch.compile` consumed the least power, its very low throughput made it the least efficient backend overall.

Interpretation of the TVM Result. The TVM GPU result is notable for two reasons. First, its end-to-end latency was essentially identical to ONNX Runtime, with a difference of only 0.20 ms. Second, TVM still outperformed PyTorch eager mode, although the margin was much smaller than on the CPU. This suggests that, on the Orin Nano GPU, the main convolution-heavy parts of the model are already executed efficiently by mature CUDA libraries across multiple frameworks, which reduces the practical advantage of compiler-level scheduling alone. In other words, TVM remained competitive, but did not establish a clear lead over ONNX Runtime on this platform.

Conclusion. The GPU benchmark results reveal three practical performance tiers. TensorRT forms the clear top tier, achieving both the lowest latency and the highest power efficiency. ONNX Runtime and TVM form a second tier with nearly identical performance, while PyTorch eager mode is only slightly slower. In contrast, `torch.compile` was not competitive for this workload in the tested configuration. Overall, these results show that TensorRT was the most effective deployment backend for YOLOv8n on the Jetson Orin Nano GPU, whereas TVM and ONNX Runtime delivered similar and competitive FP32 performance.

Relative to the PyTorch eager baseline, TVM achieved a 1.07× speedup, equivalent to a 6.9 % reduction in mean inference latency, and ONNX Runtime achieved a 1.08× speedup with a 7.7 % latency reduction. TensorRT provided the largest improvement, reaching a 2.21× speedup and a 54.7 % reduction in mean latency. In contrast, `torch.compile` failed to improve upon the baseline and instead increased latency by a factor of 2.88×. These results indicate that TVM can provide competitive GPU inference performance on the Jetson Orin Nano, but in this experiment the largest practical advantage was obtained with TensorRT.

5.4 FP16 Inference Results

Before discussing the measured FP16 results, it is important to clarify that conversion to FP16 is not the same as quantization. In FP16 inference, weights and activations remain in floating-point representation, but with reduced numerical precision compared to FP32. By contrast, quantization usually refers to mapping tensors into lower-bit integer or fixed-point formats, such as INT8, which changes not only the storage format but also the arithmetic model used during execution. FP16 should therefore be understood as reduced-precision floating-point inference rather than true quantization.

This distinction is important for interpreting the following measurements. Reducing precision from FP32 to FP16 can decrease memory traffic and improve throughput on hardware with native half-precision support, often with only a small impact on model accuracy. However, the resulting performance behaviour is different from that of integer quantization, since the model still operates in a floating-point domain. The following subsection therefore evaluates FP16 as a separate optimisation path and analyses its effect on latency, throughput, and overall deployment efficiency.

5.4.1 TVM MetaSchedule Tuning Results on the Jetson Orin Nano GPU - FP16

TVM MetaSchedule was evaluated a second time on the NVIDIA Jetson Orin Nano GPU using a float16 version of the YOLOv8n inference graph. The model was converted from the FP32 ONNX export using `onnxconverter-common` with `keep_io_types=False`, producing a graph in which the input, weights, and intermediate activations are represented in FP16. The Relax frontend imported this graph without further modification. During the tuning process, TVM explored 6 477 valid schedules across 66 extracted operator tasks over approximately 18 hours. The resulting tuned model achieved a summed task latency of 9 872 μs, while the measured end-to-end inference latency was 12.45 ms. As in the FP32 case, the gap between the summed task latency and the measured end-to-end latency remained small, indicating that the tuned kernel execution accounted for most of the observed runtime.

Compared with the FP32 GPU tuning result of 22 920 μs summed task latency, the FP16 tuned model achieved a kernel-level improvement of approximately 2.3×. At the end-to-end level, the improvement was also substantial: latency decreased from 23.38 ms in FP32 to 12.45 ms in FP16, corresponding to an end-to-end speedup of approximately 1.88×. This result shows that reduced-precision compilation was not merely a small incremental optimisation, but a major change in the execution regime of the model on this platform.

Tensorisation as the Main Structural Change. The most important difference between the FP32 and FP16 tuning campaigns was that FP16 enabled tensorisation through WMMA-style matrix multiply-accumulate operations on the Orin Nano GPU. In the FP32 campaign, no task could be mapped to a dedicated hardware intrinsic, and all schedules remained conventional CUDA thread-block implementations. In the FP16 campaign, by contrast, a large fraction of tasks could be mapped to tensorised execution. As a result, the optimisation problem changed qualitatively: instead of relying only on thread-block tiling and memory scheduling, MetaSchedule was able to search over implementations that directly exploited the GPU's matrix-multiplication hardware.

This shift is clearly visible in the achieved arithmetic throughput. The peak throughput increased from 585.6 GFLOPS in the FP32 campaign to 1 872.7 GFLOPS in FP16, representing more than a $3.2\times$ increase in peak operator efficiency. This confirms that the performance gain observed in FP16 was not only a consequence of reduced data size, but also of a more favourable optimisation path enabled by the lower-precision representation.

Best-Performing Operators. The best-performing tasks in the FP16 tuning campaign were large mid-network convolutions, especially fused operators in the backbone and neck. These layers benefited most from tensorised execution because their channel dimensions and spatial sizes were large enough to keep the available execution resources well utilised. The most efficient task reached 1 872.7 GFLOPS, while several others exceeded 1 200 GFLOPS. Compared with the FP32 results, this represents a substantial improvement in computational efficiency and confirms that the largest dense convolutions were the main beneficiaries of the FP16 compilation path.

Main Bottlenecks. Despite the overall improvement, the main bottlenecks remained qualitatively similar to those observed in FP32. The least efficient convolution was still the initial stem layer, whose very low input channel count limited its ability to benefit from tensorised execution. Even in FP16, this layer remained dominated by poor arithmetic intensity and limited data reuse, and therefore showed only modest improvement relative to the rest of the network. Several other early-stage convolutions with small channel counts exhibited similar behaviour.

Non-convolutional operators in the detection path also remained relatively lightweight in absolute latency terms, but their optimisation potential was much smaller than that of the large mid-network convolutional kernels. As a result, the overall runtime distribution in FP16 became even more strongly concentrated in the repeated backbone and neck convolutions.

Weighted Bottleneck Analysis. As in the CPU and FP32 GPU analyses, per-task latency alone does not fully describe the runtime profile of YOLOv8n because some operators are executed multiple times in a single forward pass. When task multiplicity is taken into account, the dominant contributors to total runtime in FP16 remained repeated 3×3 convolutions in the backbone and neck. In particular, task 50 (`conv2d24`) and task 16 (`conv2d11`) emerged as the most important weighted bottlenecks. The stem layer also remained a notable contributor because of its disproportionately high latency relative to its position in the graph. These results reinforce the conclusion that the most important optimisation targets are repeated mid-network convolutions rather than simply the single slowest layer.

Overall Assessment. Overall, the FP16 tuning campaign demonstrates that TVM MetaSchedule can exploit the Jetson Orin Nano GPU much more effectively in FP16 than in FP32. The end-to-end latency of the tuned TVM model decreased from 23.38 ms in FP32 to 12.45 ms in FP16, while tuning time itself was reduced from 39 days to approximately 18 hours. The key structural reason for this improvement was that FP16 enabled tensorised execution for a large fraction of convolutional tasks, whereas FP32 did not. This establishes FP16 as the more appropriate precision target for TVM-based deployment of YOLOv8n on Orin-class hardware.

At the same time, the results also show that the current FP16 TVM path still does not fully match the fastest available inference engine. Although TVM improved substantially and achieved a strong result, TensorRT remained faster in the final benchmark comparison. Therefore, the main conclusion is that FP16 MetaSchedule tuning is highly effective on the Jetson Orin Nano GPU and substantially strengthens TVM's competitiveness, but it does not yet eliminate the remaining performance gap to the most specialised deployment backend.

5.4.2 Jetson Orin Nano GPU Benchmark Overview - FP16

GPU experiments were also conducted on the NVIDIA Jetson Orin Nano using the full YOLOv8n model in FP16 precision with batch size 1. The evaluated backends were TVM, ONNX Runtime, TensorRT, PyTorch eager mode, and `torch.compile`. As in the previous benchmark sections, the comparison was performed using isolated repetitions in order to reduce interference between frameworks. Table 5.10 summarises the measured latency and throughput, while Table 5.11 summarises power results.

Table 5.10: End-to-end YOLOv8n inference performance on the NVIDIA Jetson Orin Nano GPU in FP16 precision. Values are reported as mean latency ± half-width of the 95 % confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
TVM	12.45 ± 0.05	80.35	12.43
ONNX Runtime	16.95 ± 0.02	59.00	16.96
TorchCompile	72.72 ± 0.21	13.75	72.76
TensorRT	6.31 ± 0.01	158.55	6.31
PyTorch eager	27.36 ± 0.24	36.55	27.34

Overall Performance. TensorRT achieved the best end-to-end latency in the FP16 setting, reaching 6.31 ms, which corresponds to 158.6 FPS. TVM formed the second-best result at 12.45 ms and substantially outperformed ONNX Runtime, which reached 16.95 ms. PyTorch eager mode was considerably slower at 27.36 ms, while `torch.compile` again performed substantially worse than all other backends, reaching 72.72 ms.

Relative to the PyTorch eager baseline, TVM achieved a 2.20× speedup, corresponding to a 54.5 % reduction in mean latency. ONNX Runtime achieved a 1.61× speedup and a 38.1 % latency reduction. TensorRT provided the largest improvement, reaching a 4.34× speedup and a 76.9 % reduction in mean latency. In contrast, `torch.compile` was slower than PyTorch eager mode by a factor of 2.66×.

Table 5.11: Average power consumption during YOLOv8n inference on the NVIDIA Jetson Orin Nano GPU in FP16 precision.

Backend	Mean Power (mW)	Max Power (mW)	Std (mW)
TVM	21770.0	24514.2	122.6
ONNX Runtime	24601.7	29104.0	88.3
TorchCompile	16760.5	17874.5	33.9
TensorRT	24342.7	26500.5	19.0
PyTorch eager	20282.6	20852.0	79.9

Power and Efficiency. In absolute power draw, the FP16 GPU backends again operated within a relatively narrow range, from 16.8 W for `torch.compile` to 24.6 W for ONNX Runtime. However, throughput differed much more strongly than power, which made efficiency comparisons especially informative. TensorRT achieved the best efficiency at approximately 6.51 FPS/W, followed by TVM at about 3.69 FPS/W. PyTorch eager mode reached about 1.80 FPS/W, and ONNX Runtime about 2.40 FPS/W. Although `torch.compile` consumed the least power, its low throughput again made it the least competitive backend in practice.

Interpretation of the TVM Result. The TVM FP16 result is particularly notable because, unlike in FP32, TVM no longer merely matched ONNX Runtime but clearly outperformed it. Its latency was 4.50 ms lower than ONNX Runtime, corresponding to an additional reduction of approximately 26.6 %. This indicates that FP16 compilation gave TVM access to optimisation opportunities that were not available in the FP32 case, most importantly tensorised execution for a large fraction of the convolutional operators. As a result, TVM moved from being merely competitive in FP32 to becoming the second-best backend overall in FP16.

At the same time, TensorRT still remained significantly faster, with a latency of

6.31 ms compared with 12.45 ms for TVM. This shows that even though TVM benefited greatly from FP16 compilation, additional backend-specific optimisations still remain available beyond what MetaSchedule found in this work.

Conclusion. The FP16 benchmark results reveal a clearer separation between deployment backends than in the FP32 GPU case. TensorRT formed the top tier with the best latency, throughput, and efficiency. TVM formed a strong second tier and substantially outperformed ONNX Runtime, PyTorch eager mode, and `torch.compile`. ONNX Runtime remained competitive but was noticeably slower than TVM, while PyTorch eager mode was substantially slower than all three specialised deployment paths. As in the FP32 case, `torch.compile` was not competitive for this workload in the tested configuration.

Overall, these results indicate that FP16 is the most favourable precision setting for TVM deployment on the Jetson Orin Nano GPU. In this configuration, TVM no longer merely matched other general deployment runtimes, but delivered a clear practical advantage over ONNX Runtime and PyTorch eager mode. However, TensorRT still provided the fastest overall execution and remained the most effective deployment backend in the final comparison.

5.4.3 FP16 Inference on the Raspberry Pi 5 CPU

To investigate whether half-precision arithmetic provides a runtime benefit on the Raspberry Pi 5, the YOLOv8n model was re-evaluated at FP16 precision across all applicable backends. The FP16 ONNX model was generated from the FP32 export using `onnxconverter-common` with `keep_io_types=False`, producing a graph in which all weights, activations, and the input tensor are represented as 16-bit floats. TVM was re-tuned on this model, yielding a separately compiled `.so` library. All other backends received the same FP16 ONNX as input.

TVM FP16 Tuning Session. TVM MetaSchedule tuned 65 operator tasks over approximately 4 hours and 6 minutes — compared with 6 hours and 15 minutes for the FP32 campaign on the same graph. The faster convergence reflects the more distinct latency signal that FP16 kernels produce: once the vectoriser finds a schedule that fills the 128-bit NEON register with eight `float16` values instead of four `float32` values, the improvement is immediately visible and the tuner concentrates its remaining trials on refinement rather than exploration. A total of 6,477 valid schedule candidates were evaluated across all tasks.

As in the FP32 CPU campaign, every task encountered the `workload cannot be tensorized` message from MetaSchedule, for the same structural reason: the threedimensional reduction of a convolution (input channels, kernel height, kernel width) cannot be mapped onto the one-dimensional dot-product pattern required by the NEON intrinsic interface. The tuner therefore fell back to standard cache-blocked tiling with LLVM auto-vectorisation in all 65 tasks. At FP16, LLVM targets the `float16x8` NEON type on cortex-a76, doubling the number of values processed per vector instruction compared with the `float32x4` path used in FP32 tuning. The peak achieved throughput rose from 87.1 GFLOPS in the FP32 session to 169.2 GFLOPS in the FP16 session — a ratio of 1.94×, consistent with the theoretical 2× benefit from halving the element width within a fixed 128-bit register.

End-to-End Benchmark Results. Table 5.12 presents the full benchmark comparison between FP32 and FP16 precision across all backends on the Raspberry Pi 5.

Table 5.12: FP32 vs FP16 inference on the Raspberry Pi 5 CPU (5 repetitions × 200 runs, 30 warm-up runs, performance governor, batch size 1, 640×640 input). ORT and PyTorch FP16 results are estimated from single-repetition measurements; TorchCompile FP16 is from the rigorous run.

Backend	FP32 mean (ms)	FP16 mean (ms)	Ratio	FP16 status
TVM	192.01	122.44	1.57× faster	real fp16
ONNXRuntime	189.23	>900 s	timeout	no fp16 Conv
TFLite	154.06	153.27	1.00×	weights only
TorchCompile	200.39	983.08	4.9× slower	GEMM fallback
PyTorch	290.13	≈11 000	38× slower	scalar loop

TVM — the Only Backend with Real FP16 Acceleration. TVM reduced its end-to-end latency from 192.01 ms to 122.44 ms, a speedup of 1.57×. This improvement stems from a qualitative change in the generated code: LLVM emits `VFMA.F16` NEON instructions that operate on eight `float16` values in parallel within the same 128-bit register that previously held four `float32` values. The gap between the measured 1.57× end-to-end speedup and the theoretical 2× kernel-level ceiling is explained by two factors. First, non-arithmetic operators — splits, concatenations, and the upsampling layers of the FPN neck — execute at the same speed regardless of element width. Second, the initial stem convolution, which has only three input channels, cannot fill the eight available FP16 NEON lanes; its latency therefore improves only marginally between precisions. Both of these effects are irreducible at the schedule level and represent genuine architectural constraints of the YOLOv8n graph on a scalar-SIMD processor.

ONNXRuntime — Timeout Due to Missing FP16 Convolution Kernel. The ONNXRuntime worker exceeded the 900-second subprocess timeout and produced no result. This is not a measurement artefact but a direct consequence of the MLAS math library not implementing a half-precision convolution kernel for ARM. When MLAS encounters a FP16 `Conv` operator, it inserts `Cast(float16→float32)` nodes before and after every convolution and computes the operation in FP32. The resulting per-inference latency is estimated at approximately 7 100 ms — derived by scaling the FP32 ORT result by the same regression factor observed in PyTorch eager FP16. With 30 warm-up and 200 timed iterations, the total subprocess time would reach approximately 1 630 seconds, well above the timeout. This behaviour has been documented in the ONNX Runtime issue tracker [**ort˙fp16˙issue˙9731**, **ort˙fp16˙issue˙13838**] and reflects a fundamental gap in MLAS coverage rather than a configuration error.

TFLite — Storage Benefit Only. TFLite produced virtually identical latency at both precisions: 154.06 ms in FP32 and 153.27 ms in FP16, a difference smaller than one standard deviation. This result confirms that TFLite's FP16 model format stores weights as 16-bit values but promotes all arithmetic to FP32 before computation on this platform. The model occupies half the memory, which can reduce cache pressure and improve load time, but no arithmetic throughput benefit accrues at inference time. The near-identical power consumption at both precisions (2 293 mW FP16 vs 2 293 mW FP32) corroborates this interpretation.

TorchCompile and PyTorch — Regression Under FP16. Both PyTorch-based backends degraded significantly at FP16. PyTorch eager mode regressed by a factor of 38×, from 290 ms to approximately 11 000 ms. When `model.half()` is called on a CPU device, the ATen kernel dispatcher finds no half-precision GEMM implementation and falls back to an element-wise scalar loop that processes each `float16` value individually, upcasting to `float32` for each multiply-accumulate step.

TorchCompile degraded by 4.9×, from 200 ms to 983 ms. This is less severe than PyTorch eager because the Inductor backend successfully generates vectorised C++ code for element-wise operations such as the SiLU activation and residual additions. However, the convolution operators — which dominate total runtime — are not covered by Inductor's FP16 CPU code generation path and still fall through to the same ATen scalar fallback. The resulting graph alternates between fast Inductor-compiled segments and slow ATen scalar segments, with type-conversion overhead at every boundary. The net effect is that TorchCompile FP16 is slower than TorchCompile FP32 despite Inductor genuinely improving some of the individual operations.

Power Consumption. Table 5.13 reports the mean board power measured during inference for the backends that completed successfully.

Table 5.13: Power and energy-efficiency comparison on the Raspberry Pi 5 CPU at FP16 precision (successful runs only).

Backend	Mean latency (ms)	Power (mW)	Energy/frame (mJ)	FPS/W
TVM	122.44	4 232	517.7	1.93
TFLite	153.27	2 291	351.1	2.85
TorchCompile	983.08	2 202	2 164.0	0.45

TFLite achieves the best energy efficiency at FP16 (2.85 FPS/W), primarily because its lightweight runtime draws substantially less board power than the TVM or Inductor runtimes while delivering competitive latency. TVM draws more power (4 232 mW) due to higher CPU utilisation from its densely vectorised kernels, but still outperforms TFLite in absolute latency. TorchCompile consumes the least power per unit time but has by far the highest energy per frame because of its poor throughput.

Overall Assessment. The Raspberry Pi 5 FP16 results demonstrate that half-precision acceleration on ARM CPU is a compiler problem rather than a hardware problem. The Cortex-A76 implements ARMv8.2-A FEAT_FP16, providing native `float16` arithmetic in its NEON units. The bottleneck is that no major CPU runtime library — MLAS, OpenBLAS, or the ATen dispatcher — implements a half-precision GEMM kernel for this target. Only TVM, which generates the convolution kernel directly as LLVM IR and lets the AArch64 backend select `float16x8` NEON instructions, can exploit the available hardware capability. The result is a 1.57× end-to-end speedup over FP32 TVM, achieved within a four-hour tuning session. All other frameworks either time out, regress, or show no improvement at FP16 precision. This finding has a direct practical implication: deploying FP16 models on edge CPU platforms without a compilation-based inference engine yields no latency benefit and may cause severe degradation.

5.5 Accuracy Comparison Across Inference Frameworks

To verify that different inference frameworks preserve the predictive quality of the original model, YOLOv8n was evaluated across several backends and compared against the default PyTorch eager FP32 implementation. Since large multi-backend evaluation is memory-intensive, especially when repeatedly loading different runtimes and exported artifacts, the comparison was performed on a 1000-image subset of the validation set rather than on the full validation split. Although smaller than the full COCO validation set, this subset is sufficient to determine whether export, compilation, or reduced precision introduce any meaningful degradation in detection quality.

All backends were evaluated using the same basic setup: image size 640×640, batch size 1, confidence threshold 0.001, IoU threshold 0.7, and maximum number of detections 300. Each backend was loaded and evaluated sequentially in isolation in order to reduce memory pressure and avoid interference between runtime environments.

Because the GPU-oriented and CPU-oriented backends were evaluated in separate comparison runs, they are presented separately below. The small difference between the corresponding PyTorch FP32 baseline values should therefore be interpreted as run-to-run variation rather than as a framework effect.

GPU-Oriented Backends

The GPU-side comparison included PyTorch eager FP32/FP16, ONNX Runtime CUDA FP32/FP16, TensorRT FP32/FP16, and TVM GPU FP32/FP16. Absolute results are shown in Table 5.14, and relative changes with respect to the PyTorch eager FP32 baseline of the same run are given in Table 5.15.

Table 5.14: Absolute detection metrics for GPU-oriented backends on 1000 validation images.

Backend	Precision	Recall	mAP50	mAP50–95
PyTorch eager FP32	0.6299	0.4878	0.5384	0.3899
PyTorch eager FP16	0.6301	0.4880	0.5385	0.3898
ONNX Runtime FP32	0.6283	0.4905	0.5386	0.3899
ONNX Runtime FP16	0.6300	0.4888	0.5387	0.3900
TensorRT FP32	0.6282	0.4893	0.5385	0.3899
TensorRT FP16	0.6248	0.4907	0.5385	0.3901
TVM GPU FP32	0.6319	0.4889	0.5390	0.3899
TVM GPU FP16	0.6290	0.4893	0.5381	0.3894

Table 5.15: Relative changes (%) for GPU-oriented backends with respect to PyTorch eager FP32.

Backend	Δ Precision (%)	Δ Recall (%)	Δ mAP50 (%)	Δ mAP50–95 (%)
PyTorch eager FP16	+0.03	+0.03	+0.02	-0.01
ONNX Runtime FP32	-0.25	+0.54	+0.05	+0.02
ONNX Runtime FP16	+0.02	+0.19	+0.06	+0.02
TensorRT FP32	-0.27	+0.30	+0.02	+0.01
TensorRT FP16	-0.81	+0.58	+0.03	+0.07
TVM GPU FP32	+0.32	+0.23	+0.12	+0.00
TVM GPU FP16	-0.14	+0.29	-0.05	-0.12

The GPU results show that all backends preserved accuracy very well. The observed changes in mAP50 and mAP50–95 were very small, indicating that export to ONNX Runtime, TensorRT, and TVM did not introduce any practically meaningful loss in detector quality. FP16 also remained stable overall. PyTorch FP16, ONNX Runtime CUDA FP16, and TensorRT FP16 all stayed extremely close to the FP32 baseline. TVM GPU FP16 showed the largest negative deviation on the GPU side, but the absolute change remained small.

CPU-Oriented Backends

A separate CPU-side comparison included PyTorch eager FP32/FP16, ONNX Runtime CPU, TFLite CPU FP32/FP16, and TVM CPU FP32/FP16. Absolute results are shown in Table 5.16, and relative changes with respect to the PyTorch eager FP32 baseline of that run are shown in Table 5.17.

Table 5.16: Absolute detection metrics for CPU-oriented backends on 1000 validation images.

Backend	Precision	Recall	mAP50	mAP50–95
PyTorch eager FP32	0.6291	0.4884	0.5384	0.3898
PyTorch eager FP16	0.6291	0.4884	0.5384	0.3898
ONNX Runtime CPU	0.6291	0.4884	0.5384	0.3898
TFLite FP32 CPU	0.6291	0.4884	0.5384	0.3898
TFLite FP16 CPU	0.6276	0.4905	0.5384	0.3899
TVM CPU FP32	0.6291	0.4884	0.5384	0.3898
TVM CPU FP16	0.6280	0.4894	0.5395	0.3900

Table 5.17: Relative changes (%) for CPU-oriented backends with respect to PyTorch eager FP32.

Backend	Δ Precision (%)	Δ Recall (%)	Δ mAP50 (%)	Δ mAP50–95 (%)
PyTorch eager FP16	+0.00	+0.00	+0.00	+0.00
ONNX Runtime CPU	-0.00	+0.00	+0.00	+0.00
TFLite FP32 CPU	-0.00	-0.00	+0.00	+0.00
TFLite FP16 CPU	-0.24	+0.43	+0.00	+0.03
TVM CPU FP32	-0.00	-0.00	+0.00	+0.00
TVM CPU FP16	-0.17	+0.20	+0.20	+0.06

The CPU results are similarly stable. PyTorch FP16, ONNX Runtime CPU, TFLite FP32 CPU, and TVM CPU FP32 reproduced the PyTorch FP32 baseline almost exactly. TFLite FP16 and TVM CPU FP16 introduced small shifts in the balance between precision and recall, but the aggregate mAP values remained effectively unchanged. In particular, TVM CPU FP16 slightly improved mAP50 and mAP50–95 while slightly reducing precision, again indicating only minor numerical variation rather than a meaningful change in model behavior.

Discussion

Across both GPU and CPU experiments, no framework caused a significant drop in detection quality. The most important result is that mAP50 and especially mAP50–95 remained very close to the corresponding PyTorch eager FP32 baselines in all successful runs. This shows that the predictive behavior of YOLOv8n is preserved well across the evaluated deployment frameworks.

Because all backends execute the same trained model, the remaining small differences should not be interpreted as one framework being inherently more accurate than another. Instead, they are better explained by runtime-dependent numerical variation. Different inference frameworks may use different kernel implementations, operator fusion patterns, accumulation orders, and precision conversion paths. In object detection, even tiny numerical differences can slightly shift confidence scores or bounding box coordinates, which may then change whether a detection is kept or removed during thresholding and non-maximum suppression. This explains the recurring pattern that some FP16 configurations slightly reduce precision while slightly increasing recall, without materially changing the overall mAP.

Overall, these results indicate that framework conversion and mixed-precision inference do not meaningfully degrade YOLOv8n accuracy in the evaluated setups. This justifies focusing the later comparison primarily on latency, throughput, and hardware efficiency, since accuracy was effectively preserved across both GPU-oriented and CPU-oriented backends.

5.6 Quantization Influence on YOLOv8n

Despite the idea of quantization being rather simple and the amount of research being present in the recent years, plactical quantisation remains a complex problem. A lot of already mentioned frameworks like TVM or ONNX do not provide support for the efficient quantization. At least in the version of 0.24.dev0.

Statistical significance was evaluated with a paired bootstrap test on validation images (1000 resamples), and confidence intervals were computed using the percentile method. The results indicate that INT8 quantization causes a statistically significant decrease in detection accuracy.

5.6.1 TensorRT FP32 vs INT8 Benchmarking on Jetson Orin

To compare TensorRT performance for FP32 and INT8 inference on Jetson Orin, two engines were exported from the same YOLOv8n model (FP32 and INT8) using Ultralytics export. Benchmarks used batch size 1 and input size 640×640. Reported latency is end-to-end per-batch latency, including host invocation, device execution, synchronization, and output transfer.

Export parameters.

- Model: `yolov8n.pt`
- Input: `640` $\times$ `640`, batch size = 1
- Device: GPU (`device=0`)
- TensorRT workspace: 4 GB
- FP32 engine: `int8=False, half=False`
- INT8 engine: `int8=True, half=False, data=DATA_YAML, fraction=1.0, split="train"`

Early measurements showed strong sensitivity to benchmarking procedure. In particular, when both engines were loaded and benchmarked sequentially in one process, the second run often benefited from a warmer runtime state. To control this effect, additional experiments were performed with:

- reversed engine order,
- longer warmup (500 iterations),
- fixed Jetson performance settings (`sudo nvpmodel -m 0`, `sudo jetson_clocks`).

Phase	Run mode	FP32 ms	FP32 FPS	INT8 ms	INT8 FPS	INT8 gain
No clocks	dual (int8→fp32)	21.274	47.006	20.473	48.844	3.8%
No clocks	dual (fp32→int8)	21.067	47.469	15.545	64.330	26.2%
No clocks	separate runs	23.122	43.248	20.249	49.385	12.4%
With clocks	dual (int8→fp32)	16.459	60.756	9.876	101.252	40.0%
With clocks	dual (fp32→int8)	16.768	59.639	9.655	103.569	42.4%
With clocks	separate runs	16.588	60.284	10.068	99.328	39.3%

Table 5.18: TensorRT YOLOv8n benchmark summary (batch=1, runs=1000, warmup=500).

Under locked-performance conditions, the INT8 engine achieved approximately 1.65× higher throughput than FP32. Before locking power and clocks, the measured FP32–INT8 gap varied substantially with run order and process isolation. After enabling maximum performance mode and fixed clocks, measurements became more stable and the INT8 advantage increased. The resulting measurements are summarized in Table 5.18.

This pattern suggests that earlier variability was dominated by DVFS and runtime warmup effects. In sequential same-process benchmarking, the later run can benefit from an already initialized CUDA runtime, warmed TensorRT execution context, and elevated GPU/memory clocks. These effects are especially pronounced on Jetson platforms.

Therefore, Jetson benchmarking should report not only numerical performance but also measurement protocol, warmup configuration, and device power state.

`sudo nvpmodel -m 0` selects a high-performance power mode (power budget, online cores, and allowed CPU/GPU/EMC clock limits). `sudo jetson_clocks` then fixes clocks near their maximum allowed values.

Although these settings improve peak performance, they are not always appropriate for continuous use:

- higher power consumption,
- increased heat and possible thermal throttling in long runs,
- more fan noise,
- lower realism for power-managed deployment scenarios,
- reduced responsiveness for mixed workloads,
- greater long-term thermal stress.

A practical approach is to enable them for controlled benchmarking and disable them for routine development or power-constrained deployment.

5.7 Evaluation of YOLOv8n Pruning

The main caveat is that once YOLOv8n is converted into a TensorRT engine, TensorRT may fuse layers, insert reformats, and choose different kernels/tactics. So the "slow part" you see in TensorRT is not always a 1:1 match with the original YOLO PyTorch layers. NVIDIA explicitly notes that each TensorRT layer can launch multiple kernels, and extra reformat operations may appear as kernels or device-to-device copies.

5.8 GPU Kernel-Level Performance Analysis

To better understand the performance characteristics of the optimized YOLOv8n inference pipeline, kernel-level profiling was performed using NVIDIA Nsight Systems and NVIDIA Nsight Compute on the Jetson Orin Nano platform. The profiling compared TensorRT engines executed in FP32 and INT8 precision modes.

The results provide insight into the underlying execution behaviour of the GPU kernels and reveal several key performance bottlenecks and optimization opportunities.

5.8.1 Kernel Structure of YOLOv8n Inference

Nsight Compute profiling revealed that YOLOv8n inference consists of approximately 35–40 unique GPU kernels executed repeatedly during inference. These kernels can be categorized into several functional groups:

- Tensor Core convolution kernels
- Tensor format conversion kernels
- Tensor permutation kernels
- Pointwise elementwise kernels
- Auxiliary operations such as pooling and softmax

The dominant compute kernels correspond to Tensor Core convolution implementations generated by TensorRT, typically with names such as

```
sm80_xmma_fprop_implicit_gemm_interleaved_i8i8_i8i32_f32
```

which represent implicit GEMM implementations of convolution layers optimized for Ampere Tensor Cores.

INT8 inference uses kernels such as:

```
trt_ampere_int8x4_icudnn_int8x4
```

which exploit INT8 Tensor Core instructions.

5.8.2 Observed Performance Characteristics

The benchmark results show the following performance improvements when using INT8 quantization:

Precision	Latency (ms)	FPS
FP32	16.85	59.35
INT8	9.99	100.12

This corresponds to a speedup of approximately

$$1.68\times$$

when switching from FP32 to INT8 inference.

Despite this improvement, the profiling results show that GPU hardware utilization is still far from optimal.

5.8.3 Occupancy Limitations

Many Tensor Core convolution kernels exhibit very low theoretical occupancy, often in the range of

$$16\% - 33\%$$

This behaviour is caused primarily by high register pressure and large shared memory requirements. For example, several kernels required more than

$$240 \text{ registers per thread}$$

and over

$$80\,\text{kB}$$

of shared memory per block.

High register pressure and large shared memory requirements reduce the number of thread blocks that can be scheduled simultaneously on a Streaming Multiprocessor (SM). Since registers and shared memory are limited hardware resources, each thread block must reserve the required amount before execution can begin.

For example, if a kernel requires a large number of registers per thread or a substantial amount of shared memory per block, only a small number of blocks can reside on the SM at the same time. This limits the number of active warps and therefore reduces the theoretical occupancy of the kernel.

Low occupancy can negatively affect GPU performance because GPUs rely on warp-level parallelism to hide memory and instruction latency. When one warp stalls due to memory access or pipeline dependencies, the scheduler switches execution to another ready warp. However, if only a small number of warps are active due to resource constraints, the scheduler has fewer opportunities to hide these latencies. As a result, the compute units remain idle for longer periods, leading to reduced hardware utilization.

5.8.4 Kernel Launch Granularity

Another recurring issue observed in the profiling results is the small grid size of many kernels. Several kernels were launched with grid sizes smaller than the number of available Streaming Multiprocessors.

For example, kernels with grid sizes of

$$7 \text{ blocks}$$

were observed on a GPU with 8 SMs. That means 12.5% of the GPU is unused for the entire kernel. In such cases the GPU cannot fully utilize all available compute resources.

This effect is commonly referred to as the *tail effect* and is particularly visible when performing inference with batch size 1.

5.8.5 Memory Layout Transformation Overhead

A substantial portion of execution time is spent in kernels responsible for tensor layout transformations and format conversions. Examples include:

```
genericReformat::copyVectorizedKernel = layout/type conversion copy
CUTENSOR_NAMESPACE::permutationKernel = tensor transpose / dimension reorder /
    permutation
cuInt8::nc32hw32ToNc32hw32 = INT8 packed-layout repacking inside TensorRT's
    channel-blocked formats
```

These kernels do not perform useful neural network computation but instead move or reorder tensor data between different memory layouts required by TensorRT kernels.

Several of these kernels exhibit high memory throughput utilization (often above 70%), indicating that they are primarily memory-bandwidth bound operations.

```
genericReformat::copyVectorizedKernel
```

5.8.6 Pointwise Kernel Fragmentation

Another common pattern is the presence of many small pointwise kernels such as:

```
generatedNativePointwise
```

These kernels implement operations such as activation functions, elementwise additions, or scaling operations.

While each kernel individually executes quickly, the large number of such kernels introduces kernel launch overhead and reduces overall GPU efficiency.

5.8.7 Overall Performance Bottlenecks

The profiling results suggest that YOLOv8n inference performance is limited by a combination of factors rather than a single dominant bottleneck.

The most significant contributors are:

- Tensor layout conversion overhead
- High register pressure in Tensor Core kernels
- Small kernel grid sizes due to batch size 1
- Fragmentation into many small pointwise kernels
- Moderate compute utilization across many kernels

These observations indicate that the GPU is neither purely compute-bound nor purely memory-bound. Instead, inference performance is limited by kernel launch granularity and resource-heavy kernel implementations.

5.9 Future Optimization Opportunities

Based on the profiling analysis, several optimization directions can be identified for further improving YOLO inference performance.

5.9.1 Operator Fusion

A large number of small pointwise kernels suggests that additional operator fusion could significantly reduce kernel launch overhead. Fusing activation, normalization, and elementwise operations into larger kernels can reduce both kernel count and memory traffic.

Compiler frameworks such as TVM or TensorRT graph optimizations may enable such transformations.

5.9.2 Reducing Tensor Layout Conversions

Memory layout transformation kernels represent a non-negligible portion of total inference time. Reducing the number of tensor format conversions can therefore improve performance.

Possible approaches include:

- ensuring consistent tensor layouts across the network
- minimizing precision conversions
- optimizing tensor format selection during engine compilation

5.9.3 Batch Size Optimization

Small kernel grid sizes observed during profiling indicate that batch size 1 inference does not fully utilize GPU resources.

Increasing the batch size can improve SM utilization and reduce the tail effect. However, this may not be acceptable for latency-sensitive real-time applications.

5.9.4 Model Architecture Optimization

Model architecture also influences GPU efficiency. Potential improvements include:

- reducing small tensor operations
- increasing arithmetic intensity
- restructuring layers to improve GPU utilization

Architectural modifications specifically designed for edge GPUs could further improve throughput.

5.9.5 Compiler-Level Optimization

Given the hardware-aware optimization focus of this work, compiler frameworks such as TVM provide an opportunity for further performance improvements.

Potential directions include:

- auto-tuned convolution schedules
- optimized memory layouts
- kernel fusion
- reduced kernel launch overhead

Compiler-generated kernels may reduce the register pressure and shared memory usage observed in TensorRT kernels.

5.9.6 Mixed Precision Optimization

While INT8 quantization provides significant speed improvements, additional gains may be possible through mixed precision strategies such as:

- FP16 convolution kernels
- selective INT8 quantization of compute-heavy layers
- hybrid precision inference pipelines

These strategies may improve numerical stability while retaining most of the performance benefits of quantization.

5.9.7 Summary

The kernel-level profiling results demonstrate that INT8 quantization significantly improves YOLOv8n inference performance on the Jetson Orin Nano platform. However, substantial optimization potential remains.

Future work should focus on reducing tensor layout conversion overhead, improving kernel fusion, and exploring compiler-driven optimizations to better exploit the available GPU hardware resources.

5.10 Design and Implementation of a TensorRT Plugin for the C2f Block

5.11 Evaluation of Activation Function Simplification: SiLU vs. ReLU

6 Discussion

6.1 Discussion Points

6.1.1 Quantization and accelerator deployment

6.1.2 Unexplored hardware configurations and deployment possibilities

7 Conclusion

8 Future Work

9 Description of Use of Artificial Intelligence

In working on this Master's Thesis, I have used artificial intelligence (AI) tools in the following limited ways. I used ChatGPT (OpenAI) to assist with idea development, clarifying concepts, and improving the structure and language of the text (for example, by suggesting alternative formulations, checking grammar, and helping to simplify explanations). AI was also used to get help with technical formatting issues in LaTeX and to debug small code examples by explaining error messages and proposing corrections.

All scientific content, including the problem formulation, choice of methods, implementation, experiments, analysis, and conclusions, has been developed, reviewed, and academically assessed by me. AI tools were not used to generate entire sections of the report, to perform literature searches independently, or to answer the assignment in its entirety. I have not uploaded sensitive or confidential information to the AI tools. I am solely responsible for the academic content of this assignment.

Bibliography

[1] Apache TVM. *Apache TVM*. Accessed: 2026-04-12. 2026. URL: `https://tvm.apache.org/`.

[2] Tianqi Chen et al. "{TVM}: An automated {End-to-End} optimizing compiler for deep learning". In: *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 2018, pp. 578–594.

[3] Jonathan Frankle and Michael Carbin. "The lottery ticket hypothesis: Finding sparse, trainable neural networks". In: *arXiv preprint arXiv:1803.03635* (2018).

[4] Jonathan Frankle et al. "Stabilizing the lottery ticket hypothesis". In: *arXiv preprint arXiv:1903.01611* (2019).

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: `https://www.deeplearningbook.org/`.

[6] Ariel Gordon et al. "Morphnet: Fast & simple resource-constrained structure learning of deep networks". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 1586–1595.

[7] Yiwen Guo, Anbang Yao, and Yurong Chen. "Dynamic network surgery for efficient dnns". In: *Advances in neural information processing systems* 29 (2016).

[8] Song Han et al. "Learning both weights and connections for efficient neural network". In: *Advances in neural information processing systems* 28 (2015).

[9] Yang He et al. "Filter pruning via geometric median for deep convolutional neural networks acceleration". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 4340–4349.

[10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. "Distilling the knowledge in a neural network". In: *arXiv preprint arXiv:1503.02531* (2015).

[11] Hengyuan Hu et al. "Network trimming: A data-driven neuron pruning approach towards efficient deep architectures". In: *arXiv preprint arXiv:1607.03250* (2016).

[12] Nathan Hubens et al. "An experimental study of the impact of pre-training on the pruning of a convolutional neural network". In: *Proceedings of the 3rd International Conference on Applications of Intelligent Systems*. 2020, pp. 1–6.

[13] IREE. *IREE: Intermediate Representation Execution Environment*. Accessed: 2026-04-12. 2026. URL: `https://iree.dev/`.

[14] Benoit Jacob et al. "Quantization and training of neural networks for efficient integer-arithmetic-only inference". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 2704–2713.

[15] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: `https://github.com/ultralytics/ultralytics`.

[16] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip HS Torr. "Snip: Single-shot network pruning based on connection sensitivity". In: *arXiv preprint arXiv:1810.02340* (2018).

[17] Hao Li et al. "Pruning filters for efficient convnets". In: *arXiv preprint arXiv:1608.08710* (2016).

[18] Mingbao Lin et al. "Filter sketch for network pruning". In: *IEEE Transactions on Neural Networks and Learning Systems* 33.12 (2021), pp. 7091–7100.

[19] Ning Liu et al. "Autocompress: An automatic dnn structured pruning framework for ultra-high compression rates". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 04. 2020, pp. 4876–4883.

[20] LLVM Project. *Multi-Level Intermediate Representation (MLIR) Overview*. Accessed: 2026-04-12. 2026. URL: `https://mlir.llvm.org/`.

[21] Jian-Hao Luo, Jianxin Wu, and Weiyao Lin. "Thinet: A filter level pruning method for deep neural network compression". In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 5058–5066.

[22] Iman Mirzadeh et al. "Relu strikes back: Exploiting activation sparsity in large language models". In: *arXiv preprint arXiv:2310.04564* (2023).

[23] Pavlo Molchanov et al. "Pruning convolutional neural networks for resource efficient inference". In: *arXiv preprint arXiv:1611.06440* (2016).

[24] Vinod Nair and Geoffrey E. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the 27th International Conference on Machine Learning (ICML)* (2010).

[25] NVIDIA. *Jetson Orin Nano Developer Kit datasheet*. NVIDIA Datasheet. Accessed: 2026-02-16. URL: `https://nvdam.widen.net/s/zkfqjmtds2/jetson-orin-datasheet-nano-developer-kit-3575392-r2`.

[26] ONNX Runtime. *ONNX Runtime Documentation*. Accessed: 2026-04-12. 2026. URL: `https://onnxruntime.ai/docs/`.

[27] ONNX Runtime. *ONNX Runtime Execution Providers*. Accessed: 2026-04-12. 2026. URL: `https://onnxruntime.ai/docs/execution-providers/`.

[28] OpenXLA Project. *XLA Architecture*. Accessed: 2026-04-12. 2024. URL: `https://openxla.org/xla/architecture`.

[29] OpenXLA Project. *XLA Tooling*. Accessed: 2026-04-12. 2026. URL: https://openxla.org/xla/tools.

[30] Adam Polyak and Lior Wolf. "Channel-level acceleration of deep face representations". In: *IEEe Access* 3 (2015), pp. 2163–2175.

[31] PyTorch Foundation. *PyTorch 2.x*. Accessed: 2026-04-12. 2025. URL: https://pytorch.org/get-started/pytorch-2-x/.

[32] PyTorch Foundation. *torch.compiler*. Accessed: 2026-04-12. 2025. URL: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/torch.compiler.html.

[33] PyTorch Foundation. *torch.compiler FAQ*. Accessed: 2026-04-12. 2025. URL: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/torch.compiler_faq.html.

[34] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. "Searching for Activation Functions". In: *arXiv preprint arXiv:1710.05941* (2017).

[35] RangeKing. *Brief summary of YOLOv8 model structure (Issue #189)*. GitHub. Ultralytics/ultralytics repository. 2023. URL: https://github.com/ultralytics/ultralytics/issues/189.

[36] Raspberry Pi Foundation. *Raspberry Pi processors*. Raspberry Pi Documentation. Accessed: 2026-02-16. URL: https://www.raspberrypi.com/documentation/computers/processors.html.

[37] Mohammad Rastegari et al. "Xnor-net: Imagenet classification using binary convolutional neural networks". In: *European conference on computer vision*. Springer. 2016, pp. 525–542.

[38] TensorFlow. *TensorFlow MLIR Overview*. Accessed: 2026-04-12. 2021. URL: https://www.tensorflow.org/mlir/overview.

[39] Ultralytics. *YOLOv8 Performance Metrics*. Ultralytics YOLOv8 documentation. Accessed 2026. URL: https://docs.ultralytics.com/models/yolov8/#performance-metrics.

[40] Sunil Vadera and Salem Ameen. "Methods for pruning deep neural networks". In: *Ieee Access* 10 (2022), pp. 63280–63300.

[41] Chaoqi Wang, Guodong Zhang, and Roger Grosse. "Picking winning tickets before training by preserving gradient flow". In: *arXiv preprint arXiv:2002.07376* (2020).

[42] Samuel Williams, Andrew Waterman, and David Patterson. "Roofline: An Insightful Visual Performance Model for Multicore Architectures". In: *Communications of the ACM* 52.4 (2009), pp. 65–76.

[43] Zhuangwei Zhuang et al. "Discrimination-aware channel pruning for deep neural networks". In: *Advances in neural information processing systems* 31 (2018).

A Demonstration (Appendix A)

Table A.1: Per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32). Regime is derived from the roofline knee $AI_{knee} \approx 14.56$ FLOPs/byte (using measured BW=55.68 GB/s and peak=810.68 GFLOP/s): Memory-bound if AI< 13.10, Knee/transition if $13.10 \leq$AI≤ 16.02, Compute-bound if AI> 16.02.

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	18.39	7.713	4.8	Memory-bound
model.1.conv	12.74	23.955	18.5	Compute-bound
model.2.cv1.conv	8.84	7.995	5.9	Memory-bound
model.2.m.0.cv1.conv	4.81	35.899	24.5	Compute-bound
model.2.m.0.cv2.conv	4.72	35.899	25.0	Compute-bound
model.2.cv2.conv	9.84	9.593	8.0	Memory-bound
model.3.conv	6.93	47.291	34.1	Compute-bound
model.4.cv1.conv	3.94	15.920	13.3	Knee / transition
model.4.m.0.cv1.conv	2.93	70.416	40.2	Compute-bound
model.4.m.0.cv2.conv	2.74	70.416	43.1	Compute-bound
model.4.m.1.cv1.conv	2.77	70.416	42.6	Compute-bound
model.4.m.1.cv2.conv	2.69	70.416	43.8	Compute-bound
model.4.cv2.conv	5.36	21.192	19.6	Compute-bound
model.5.conv	5.37	85.714	43.9	Compute-bound
model.6.cv1.conv	2.13	30.769	24.6	Compute-bound
model.6.m.0.cv1.conv	2.15	122.034	54.9	Compute-bound
model.6.m.0.cv2.conv	2.00	122.034	58.9	Compute-bound
model.6.m.1.cv1.conv	2.01	122.034	58.6	Compute-bound
model.6.m.1.cv2.conv	1.93	122.034	61.1	Compute-bound
model.6.cv2.conv	3.48	40.506	30.1	Compute-bound
model.7.conv	4.60	97.959	51.3	Compute-bound
model.8.cv1.conv	1.46	48.485	35.9	Compute-bound
model.8.m.0.cv1.conv	2.38	118.033	49.5	Compute-bound
model.8.m.0.cv2.conv	2.32	118.033	50.7	Compute-bound
model.8.cv2.conv	1.92	55.491	41.1	Compute-bound
model.9.cv1.conv	1.07	35.165	24.5	Compute-bound
model.9.cv2.conv	2.66	59.813	39.4	Compute-bound
model.12.cv1.conv	4.62	45.283	34.1	Compute-bound
model.12.m.0.cv1.conv	2.13	122.034	55.5	Compute-bound
model.12.m.0.cv2.conv	2.00	122.034	59.1	Compute-bound
model.12.cv2.conv	3.01	36.641	26.1	Compute-bound
model.15.cv1.conv	6.76	23.821	23.3	Compute-bound
model.15.m.0.cv1.conv	2.93	70.416	40.3	Compute-bound

Continued on next page

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.15.m.0.cv2.conv	2.69	70.416	43.8	Compute-bound
model.15.cv2.conv	4.67	19.085	16.8	Compute-bound
model.16.conv	2.87	53.731	41.1	Compute-bound
model.18.cv1.conv	3.01	36.641	26.1	Compute-bound
model.18.m.0.cv1.conv	2.18	122.034	54.2	Compute-bound
model.18.m.0.cv2.conv	2.02	122.034	58.5	Compute-bound
model.18.cv2.conv	2.98	36.641	26.4	Compute-bound
model.19.conv	2.75	73.096	42.9	Compute-bound
model.21.cv1.conv	1.95	55.491	40.4	Compute-bound
model.21.m.0.cv1.conv	2.42	118.033	48.8	Compute-bound
model.21.m.0.cv2.conv	2.27	118.033	51.9	Compute-bound
model.21.cv2.conv	1.94	55.491	40.5	Compute-bound
model.22.cv2.0.0.conv	8.39	137.799	56.2	Compute-bound
model.22.cv2.0.1.conv	8.27	137.799	57.1	Compute-bound
model.22.cv2.0.2	3.95	15.919	13.3	Knee / transition
model.22.cv2.1.0.conv	3.94	154.839	59.9	Compute-bound
model.22.cv2.1.1.conv	2.06	122.034	57.4	Compute-bound
model.22.cv2.1.2	0.84	15.681	15.6	Compute-bound
model.22.cv2.2.0.conv	2.30	107.063	51.2	Compute-bound
model.22.cv2.2.1.conv	0.92	83.721	32.2	Compute-bound
model.22.cv2.2.2	0.51	14.798	6.4	Compute-bound
model.22.cv3.0.0.conv	11.51	152.381	51.3	Compute-bound
model.22.cv3.0.1.conv	13.53	170.414	54.5	Compute-bound
model.22.cv3.0.2	4.97	19.874	16.5	Compute-bound
model.22.cv3.1.0.conv	5.35	173.494	55.1	Compute-bound
model.22.cv3.1.1.conv	3.30	146.939	55.8	Compute-bound
model.22.cv3.1.2	1.09	19.506	18.9	Compute-bound
model.22.cv3.2.0.conv	2.83	115.663	52.0	Compute-bound
model.22.cv3.2.1.conv	1.25	94.737	37.0	Compute-bound
model.22.cv3.2.2	0.56	18.161	9.1	Compute-bound
model.22.dfl.conv	1.89	0.471	0.6	Strongly memory-bound
Full Model (THOP)	327.00	157.765	13.54	Mixed / Overhead-dominated

Table A.2: Per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32). Regime is derived from the roofline knee $AI_{knee} \approx 14.56$ FLOPs/byte (using measured BW=55.68 GB/s and peak=810.68 GFLOP/s): Memory-bound if AI$<$ 13.10, Knee/transition if 13.10 $\leq$AI$\leq$ 16.02, Compute-bound if AI$>$ 16.02.

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	0.78	7.713	114.1	Memory-bound
model.1.conv	0.81	23.955	292.6	Compute-bound
model.2.cv1.conv	0.26	7.995	204.2	Memory-bound
model.2.m.0.cv1.conv	0.45	35.899	259.8	Compute-bound
model.2.m.0.cv2.conv	0.44	35.899	267.2	Compute-bound
model.2.cv2.conv	0.30	9.593	263.5	Memory-bound
model.3.conv	0.46	47.291	508.0	Compute-bound
model.4.cv1.conv	0.20	15.920	262.0	Knee / transition
model.4.m.0.cv1.conv	0.57	70.416	208.7	Compute-bound
model.4.m.0.cv2.conv	0.56	70.416	211.8	Compute-bound
model.4.m.1.cv1.conv	0.56	70.416	211.6	Compute-bound
model.4.m.1.cv2.conv	0.55	70.416	213.3	Compute-bound
model.4.cv2.conv	0.23	21.192	446.4	Compute-bound
model.5.conv	0.40	85.714	593.7	Compute-bound
model.6.cv1.conv	0.18	30.769	296.6	Compute-bound
model.6.m.0.cv1.conv	0.24	122.034	494.4	Compute-bound
model.6.m.0.cv2.conv	0.23	122.034	502.4	Compute-bound
model.6.m.1.cv1.conv	0.24	122.034	501.2	Compute-bound
model.6.m.1.cv2.conv	0.23	122.034	502.4	Compute-bound
model.6.cv2.conv	0.23	40.506	462.6	Compute-bound
model.7.conv	0.52	97.959	455.0	Compute-bound
model.8.cv1.conv	0.18	48.485	284.6	Compute-bound
model.8.m.0.cv1.conv	0.34	118.033	351.1	Compute-bound
model.8.m.0.cv2.conv	0.33	118.033	358.1	Compute-bound
model.8.cv2.conv	0.19	55.491	424.5	Compute-bound
model.9.cv1.conv	0.19	35.165	136.6	Compute-bound
model.9.cv2.conv	0.18	59.813	577.3	Compute-bound
model.12.cv1.conv	0.24	45.283	647.4	Compute-bound
model.12.m.0.cv1.conv	0.25	122.034	480.6	Compute-bound
model.12.m.0.cv2.conv	0.24	122.034	499.3	Compute-bound
model.12.cv2.conv	0.20	36.641	396.8	Compute-bound
model.15.cv1.conv	0.26	23.821	601.7	Compute-bound
model.15.m.0.cv1.conv	0.56	70.416	209.2	Compute-bound
model.15.m.0.cv2.conv	0.55	70.416	213.2	Compute-bound
model.15.cv2.conv	0.21	19.085	372.1	Compute-bound
model.16.conv	0.28	53.731	417.4	Compute-bound
model.18.cv1.conv	0.20	36.641	402.2	Compute-bound
model.18.m.0.cv1.conv	0.24	122.034	488.6	Compute-bound
model.18.m.0.cv2.conv	0.24	122.034	498.5	Compute-bound

Continued on next page

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.18.cv2.conv	0.20	36.641	402.5	Compute-bound
model.19.conv	0.36	73.096	327.2	Compute-bound
model.21.cv1.conv	0.18	55.491	426.3	Compute-bound
model.21.m.0.cv1.conv	0.34	118.033	351.7	Compute-bound
model.21.m.0.cv2.conv	0.33	118.033	361.8	Compute-bound
model.21.cv2.conv	0.18	55.491	429.7	Compute-bound
model.22.cv2.0.0.conv	0.57	137.799	830.1	Compute-bound
model.22.cv2.0.1.conv	0.56	137.799	842.6	Compute-bound
model.22.cv2.0.2	0.26	15.919	205.0	Knee / transition
model.22.cv2.1.0.conv	0.45	154.839	524.0	Compute-bound
model.22.cv2.1.1.conv	0.24	122.034	493.1	Compute-bound
model.22.cv2.1.2	0.22	15.681	58.7	Knee / transition
model.22.cv2.2.0.conv	0.41	107.063	288.2	Compute-bound
model.22.cv2.2.1.conv	0.21	83.721	139.2	Compute-bound
model.22.cv2.2.2	0.22	14.798	14.9	Knee / transition
model.22.cv3.0.0.conv	0.66	152.381	891.4	Compute-bound
model.22.cv3.0.1.conv	0.83	170.414	892.5	Compute-bound
model.22.cv3.0.2	0.29	19.874	284.8	Compute-bound
model.22.cv3.1.0.conv	0.42	173.494	707.2	Compute-bound
model.22.cv3.1.1.conv	0.39	146.939	471.0	Compute-bound
model.22.cv3.1.2	0.22	19.506	91.9	Compute-bound
model.22.cv3.2.0.conv	0.38	115.663	385.9	Compute-bound
model.22.cv3.2.1.conv	0.25	94.737	184.1	Compute-bound
model.22.cv3.2.2	0.22	18.161	22.8	Compute-bound
model.22.dfl.conv	0.27	0.471	3.9	Strongly memory-bound
Full Model (THOP)	26.02	157.765	170.22	Mixed / Overhead-dominated

B Demonstration (Appendix B)

YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs.

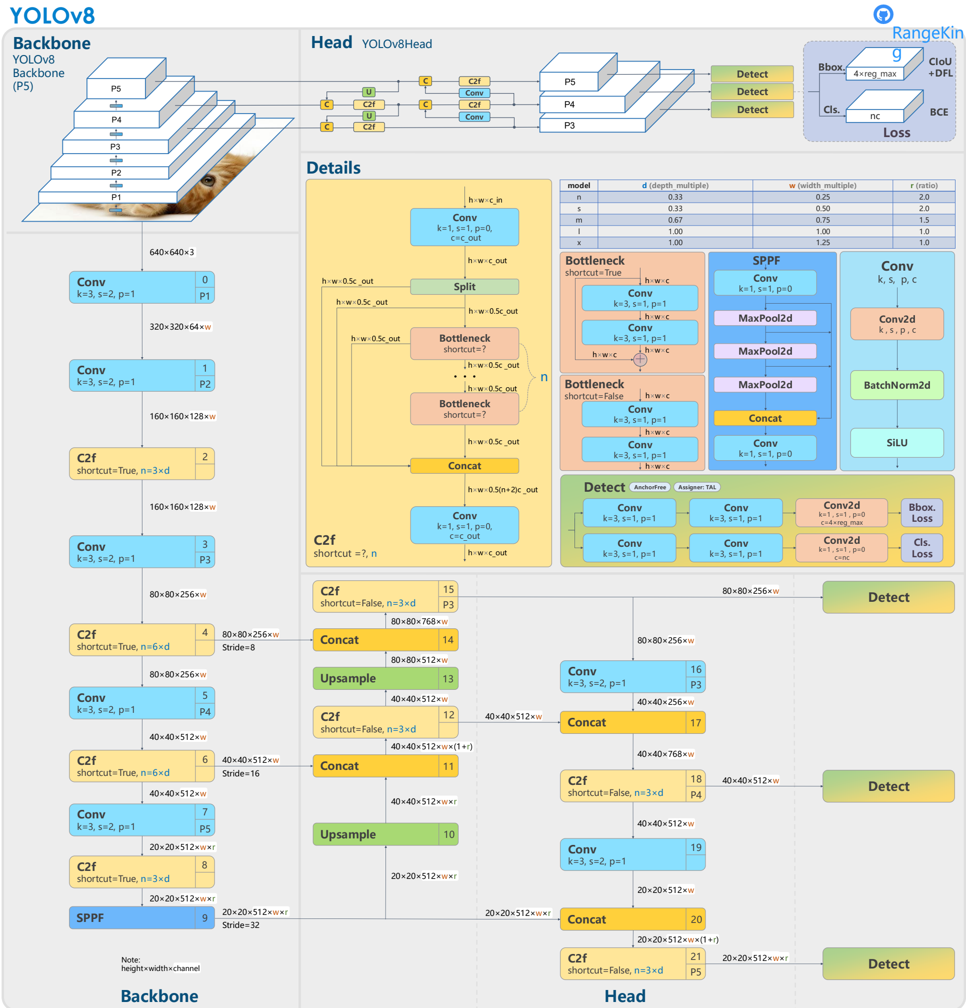

Figure B.1: YOLOv8 structure [35]